

Quantum Teleportation: Hands-On Course Notes

SIGGRAPH 2026
19 – 23 July, 2026
Los Angeles, California

Andrew Glassner
pianohands@proton.me
www.glassner.com

1 Introduction

Quantum teleportation is a fun algorithm to write and run. The purpose of the algorithm is to “teleport” (that is, transfer) the state of a quantum bit from one physical quantum qubit to another, with the help of two classical bits communicated by traditional means.

In this course we’ll do some light Python programming to run quantum teleportation on simulators. A big advantage of using simulators, rather than real hardware, is that we can read the entire state of a quantum bit at the end. With this special trick, we can use quantum teleportation to send a secret message!

These notes assume that you’re familiar with the basic ideas, terms, and notation of quantum computing. If you attended my companion SIGGRAPH course, *An Introduction to Quantum Computing*, you’re all set. Otherwise, you can learn this information from many sources. I of course recommend my book, *Quantum Computing: From Concepts to Code* [Glassner(2025)] as a starting place, particularly if you’d like a math refresher before getting into the depths.

These notes describe how to replicate the hands-on lab at home. For a discussion of the teleportation algorithm itself, please see the accompanying notes (chapter 7 from my book).

We’ll use three notebooks for this lab. All of the notebooks contain lots of comments, but necessarily not much discussion. That’s what these notes are for. You can treat this document as a kind of super-commentary for the notebooks, supplementing the comments they contain.

2 Simulators

There are real quantum computers available today, offering web-based interfaces for uploading your programs and getting back the measured results. Some vendors even offer a limited amount of time on their smaller quantum computers for free.

While the code we'll write could run on real quantum computers, in these notes we'll use free, high-quality simulators.

One of the main reasons is that the free online computers are heavily subscribed, so it can take hours before the program you submit gets to the head of the queue and is executed. In the course we couldn't wait that long. Simulators, by contrast, give us immediate results.

More importantly for this algorithm, simulators let us directly see the state of a quantum bit. That is, we can get back the two complex numbers that are the coefficients on states $|0\rangle$ and $|1\rangle$. We can't do that on real hardware, because measurement causes the qubit to collapse. All we can learn is whether it collapsed to 0 or 1. Simulators let us peek at information that we wouldn't be able to obtain in reality, which is the essential final step that allows us to communicate a secret message.

I've chosen two simulators from the IBM Qiskit (kiss'-kit) family. Qiskit is a mature and sophisticated quantum ecosystem, with simulators that are easy to use. The cherries on top are that it's all free, and the code we run on the simulators can be run, without change, on real hardware.

3 Python Environments

We'll use several free Python libraries, which you'll need to install. You can just open up a shell and install the libraries and then run the code. That will probably work just fine.

To install a library, open a shell and use the `pip` command from the command line. For example `pip install numpy` will install the `numpy` library.

In my experience, a more pleasant way to go is to create and use a Python *virtual environment*, often just called an *environment*. Environments are basically little self-contained worlds of Python and its libraries, distinct from all other worlds. This means that you can install the libraries you need for this project, and not have to worry that they might conflict with other libraries you installed in the past (or might in the future).

Python offers a free, built-in virtual environments system called *venv* (<https://docs.python.org/3/library/venv.html>). To install a library from one of these environments, use `pip` just as I described above.

I prefer the somewhat snazzier, but also free, virtual environment system `conda` from Anaconda (<https://www.anaconda.com/download>). The commands for this version all begin with `conda`. You can find lots of information online on how to install Anaconda, and create and manage virtual environments with it. I won't include all of that here because it changes from time to time, so looking it up when you need it is the best way to get up-to-date information.

To install the `numpy` library using Anaconda, you'd open a shell and enter `conda install numpy`.

Here are the libraries to install. They're all free, and they should all just install cleanly with no additional effort on your part. If something does go wrong, search online for help.

- `numpy`: great everyday math library
- `matplotlib`: make plots
- `time`: measure how long a program runs
- `jupyter`: cell-based Python development
- `pylatexenc`: a utility used by other libraries
- `qiskit`: basic quantum simulation package
- `qiskit_aer`: quantum simulators and support

If you leave a virtual environment alone for a while and later return to it, it's usually a good idea to make sure everything is still up to date. In Anaconda, type `conda update --all` from a command line. Lots of stuff might update, but that's fine. Handling that complexity for you is part of what makes virtual environments great.

4 Getting The Notebooks

We'll work with three notebooks. To keep things up to date as time moves on, I've included the current year in their names:

- `Intro-2026.ipynb`: basic quantum circuits
- `Teleport-Alice-2026.ipynb`: Alice's side of teleportation
- `Teleport-Bob-2026.ipynb`: Bob's side of teleportation

You can download them all for free. Notebooks have the standard suffix `.ipynb`, which you can think of as shorthand for “interactive Python notebook”.

To download the files, point your browser to my Github page at the address <https://github.com/blueberrymusic> and look for *Quantum Teleportation Lab*.

5 Jupyter

To write and run code, we'll be using an interactive, browser-based environment called **Jupyter**. There are several flavors of this system. We'll be using the simplest one called a *notebook*. Once you've installed the **Jupyter** library as in Section 3, open a shell and navigate to the directory where you saved the notebooks from my Github (that is, the files ending in `.ipynb`).

Now start up Jupyter from a shell by typing `jupyter notebook`. Leave the shell running; if you accidentally stop the program or close the shell window, Jupyter will hang and become unresponsive. Just close all the Jupyter-related tabs in your browser and restart things by typing `jupyter notebook` into a shell again. You'll lose any changes you've made since your last save, so try not to close the shell until you've saved your work!

After typing `jupyter notebook`, a browser window should open showing you the notebook files in your folder, perhaps with some other files (typically Jupyter will start a browser for you if you don't have one already open, but if that fails, open up a browser before starting Jupyter). It should look something like Figure 1.

If there are folders in your starting directory, you can double-click on them to open them, and otherwise treat this window as a file browser.

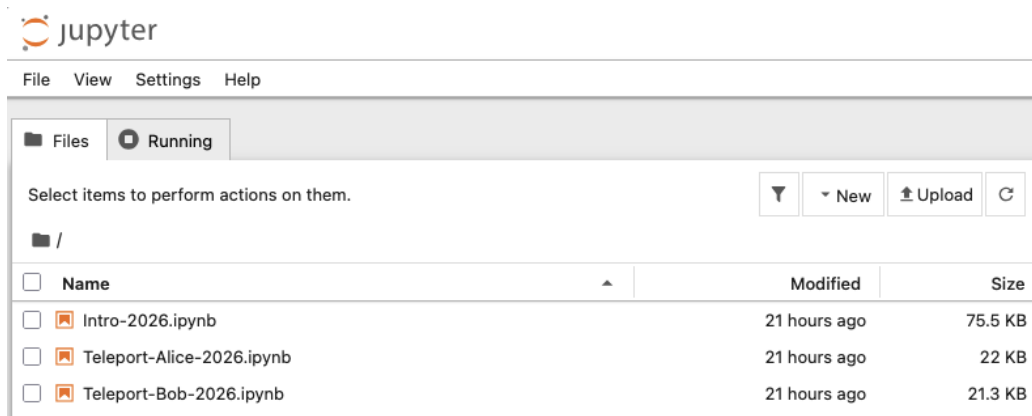


Figure 1: A typical starting Jupyter window

To open a notebook, just double-click on it. Try it now and double-click on `Intro-2026.ipynb`.

You'll see that the notebook is arranged as a series of *cells*. Each cell in our notebooks holds either a *comment* or *code*. Comments are written in the popular *Markdown* system, so you can do some basic formatting.

You *select* a cell by clicking once anywhere inside of it. You can create your own cell after the currently selected cell (or at the start of the notebook if there are no cells already there) using the plus button in the menu at the top (the one with the floppy disk icon at the far left, as shown in Figure 2). You can set the type of cell (code or Markdown, and sometimes other choices) using the drop-down menu at the right of this group of buttons, also shown in Figure 2.

You run (or execute) the code in a cell by selecting it and pressing *shift-return*. Any output will appear immediately below that cell, in an *output cell*. It's perfectly safe to execute a comment cell; nothing will happen!

You'll notice that at the top-left of each code cell there's a pair of square brackets, which initially have a space between them: `[ ]`. Once you press *shift-enter* in a cell, the brackets will hold an asterisk, like this: `[*]`. This tells you that the cell is executing. The first cell you execute in a fresh notebook often takes a few seconds to run.

When a cell is done executing, the asterisk is replaced with a number telling you where this sits in the sequence of cells you've executed. For example, the first cell you execute will be marked `[1]`, the second `[2]`, and so

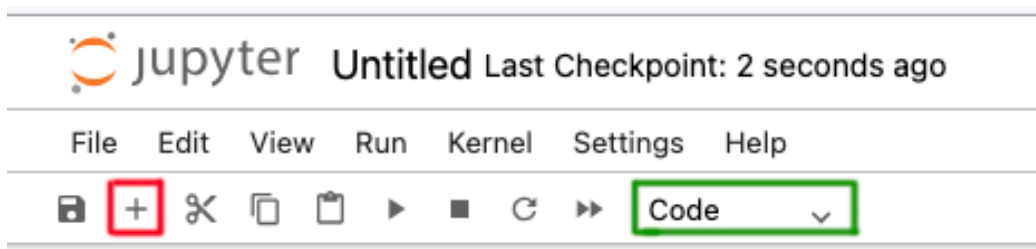


Figure 2: The red box shows the plus button that creates a new cell. The green box shows the drop-down menu lets you choose Code or Markdown

on. Any output appears directly under the cell, and the system automatically creates a new cell and selects it, ready for input. A before-and-after pair is shown in Figure 3.



Figure 3: Left: A cell before execution. Right: After execution, the cell is marked with its index to the left, its output appears immediately under it, and a new input cell is automatically created and selected, awaiting input.

Be careful, because the number shown next to each cell is completely decoupled from the linear ordering of the cells as they appear in the notebook. If you execute the first cell, then the second, and then the first cell again, the first cell's number will be updated to [3] to show that it's the third executed cell. This is a great system, because if you have a typo in a cell, or you change your mind about some settings in there, you can just edit the cell by clicking in it and typing as usual, and then hit *shift-enter* to execute it again. This is just like typing the new contents of the cell into a Python interpreter. New stuff it hasn't seen before gets added in, and changed stuff over-writes previous values.

The risk is that if you scroll up and down through the notebook, executing cells here and there out of the linear order in which they appear in the file, the system will be perfectly happy but you can get confused about what state you’ve gotten yourself into. If that happens, you can go up to the *Kernel* menu (think of “kernel” meaning “python interpreter”). The choice I use most frequently here is **Restart Kernel and Clear Outputs of All Cells...** which basically clears everything and gives you a fresh Python interpreter, and the numbers in front of all the cells are erased (because nothing has been executed yet). You can then execute cells as before. Figure 4 shows this menu.

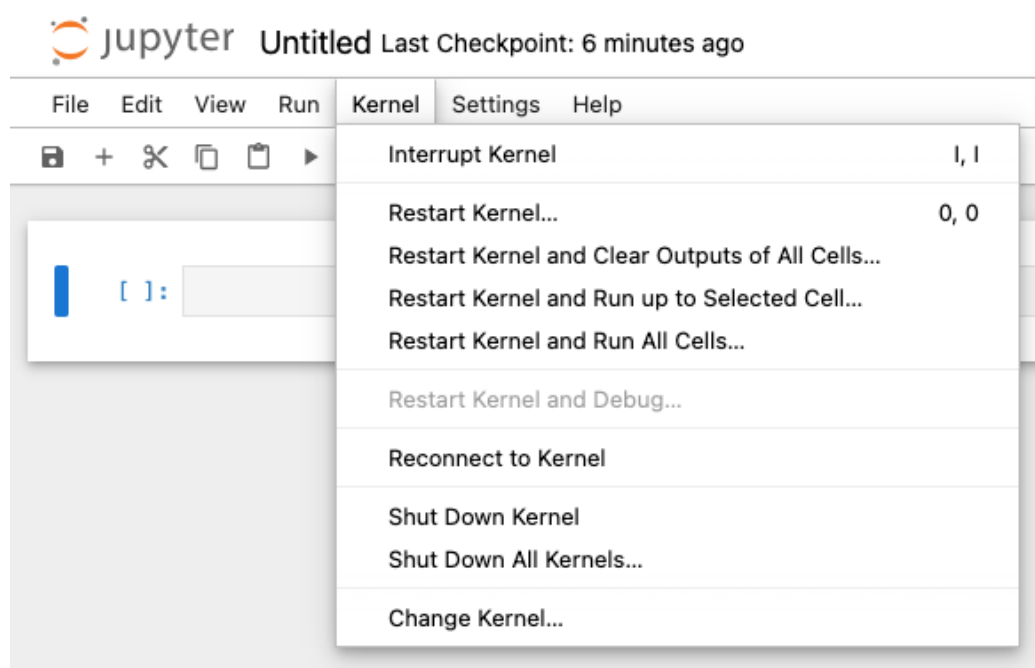


Figure 4: The Kernel menu lets you stop (or interrupt) execution of any cell, restart the kernel, and take other actions

If you want to execute a bunch of cells in sequence at once, there are lots of options for doing this in one click under the *Run* menu. I often click on the cell I’m working on and then choose **Run All Above Selected Cell** to run the cells in order, from top to bottom as they appear in the notebook, up to the current cell (if you want this to happen in a fresh version of Python,

be sure to choose one of the **Restart Kernel...** options first. Forgetting to do this can lead to some really frustrating debug sessions!). To check that the whole notebook is working right, **Run All Cells** executes them all. It will stop if one of the cells reports an error.

I wrote these notebooks so that you can execute the cells in order, top to bottom, and everything will work (though you'll want to manually set some information at the end of both of the **Teleport** notebooks; both of these steps are explained in detail later).

When you execute a cell (and it doesn't have errors), the next cell will be automatically selected. So you can click in the first cell and then just keep hitting *shift-enter* to execute the notebook in sequence, one cell at a time.

Remember that you can always create a new cell (with the **+** button) and then enter any Python code you like in there. This is a great way to learn what a notebook is doing: execute a cell, and then print out the different variables that are created or changed, dive into indexed lists, and even mess around with variables and functions.

When debugging or exploring someone else's notebook, I often make new cells after steps I'm not quite following, and use them to print things out using Python's `print()` routine (the output appears immediately below in an output cell), then click in the cell I'm stryding and change something, execute it again, execute the print cell again, and so on, modifying and then executing the same cell over and over as I dig in deeper to whatever it is I'm exploring.

You can arrange your code in cells freely any way you like; you can even ignore the cells altogether and put everything in one enormous cell if you're feeling like a renegade.

As I mentioned earlier, sometimes the first cell in a notebook can take a while to execute, particularly if it's importing a bunch of libraries. As soon as you hit *shift-enter*, the cell's label should become `[*]`. If you see that asterisk, just hang on and let it run. If you feel it's gone on way too long, go to the *Kernel* menu and choose **Interrupt Kernel**, which should stop everything. Then you can try to figure out the problem. If the cell has a lot of `import` statements, try splitting that cell up into several cells with just one (or a few) `import` calls in each, so you can figure out which one is hanging.

You can save your notebook using the *File* menu at the top of the Jupyter window. To close a notebook, just close the browser tab (remember to save first if you made changes you want to keep).

There's a lot more to say about Jupyter in general and notebooks in particular. You can find everything from intro tutorials to advanced techniques online. Jupyter isn't just for Python, and people use it for lots of other languages as well.

6 Notebook 1: Intro

Enough background, let's start writing quantum code!

We'll start with the introductory notebook, `Intro.ipynb`. It will let us check that everything is working properly, and get our feet wet by running a couple of simple circuits.

6.1 Imports and Support

This notebook starts with `import` statements to read in the packages that we'll need. Here's a brief rundown:

numpy: This is a great everyday math library, conventionally imported with the prefix `np`. It also provides lots of linear algebra functions.

matplotlib: An awkwardly-named package, this is the basic library for making graphs and plots. There are some nicer libraries (such as `seaborn`), but I've used this here for simplicity. Almost everyone imports the `pyplot` part of this library, and gives it the shorthand name `plt`.

time: A utility for measuring how long it takes for a command to run in system time (as distinct from wall-clock time).

qiskit QuantumCircuit: The `qiskit` library is IBM's simulator library. It holds a great deal of stuff. Here we import just `QuantumCircuit`, which builds a circuit from quantum gates.

qiskit transpile: Another piece from `qiskit`, this routine converts our circuit, described in Python, into a form that the simulator can execute. It's like a compiler, turning a high-level description into a low-level version.

qiskit_aer Aer: This is the simulator itself.

`qiskit.visualization plot_histogram`: This is a convenience function that lets us draw nice bar charts of the values we measure from many runs of a quantum circuit.

`%matplotlib inline`: This isn't a library, but a special directive (sometimes called a *magic* directive) to Jupyter telling it to draw graphics output in cells directly in the notebook, rather than opening a new window for them.

My goal in this notebook is to let you focus on creating and running your circuits, and not on the mechanics for setting up and running the simulator and plotting its output. So the imports are followed by a function I wrote called `run_project`, which handles all that stuff. It takes three arguments:

`make_gates`: This is the function that you write to define your quantum circuit. I initialize the circuit and then call this to insert the gates.

`num_qubits`: To keep things simple, you tell the routine how many qubits are in your circuit, rather than making it work that out.

`num_shots`: How many times your circuit should be run. In the real world, quantum computers are *noisy*, which means they make errors that aren't corrected. And some computations, like measuring a state in superposition, can be different from one run of the circuit to the next. This argument lets you say how many times your circuit should be run.

In short, `run_project` creates a circuit with the specified number of qubits, all initialized to $|0\rangle$. Then it calls your routine to insert your quantum gates into that circuit, and then adds a measurement step for each qubit. It prepares the resulting circuit for the simulator, and runs it the requested number of times. The function wraps up by printing and plotting the results.

To learn more about the simulator and its related tools, check out IBM's documentation [Qiskit(2026)].

6.2 Hello, World

Let's start with a check that everything is up and running.

The one-qubit circuit we're creating here has no quantum gates! It's an empty circuit. Since every qubit starts as $|0\rangle$ by default, and that's not a superposition, we expect that it will always be measured as a 0.

```
def make_gates_Hello_World(qc):  
    pass
```

Figure 5: Listing for Hello, World

The function that builds this empty circuit is called `make_gates_Hello_World`. It takes one argument, `qc`, which is the name of the quantum circuit that `run_project` builds. You'll see how it's used in the next program.

I've made a function for you for this program, and the ones to come. Each of these functions is called `make_gates_<name>` where `<name>` is replaced by the name of the program we're writing. I've left these functions empty so that you get to type in the commands to build the circuit. I find that even the simple act of typing in these short lines of code is a great way to get them under your fingertips and begin to get familiar with them. I'll show you exactly what to type, but it will be your job to do the typing!

Since `make_gates_Hello_World` does nothing, we need to use Python's `pass` statement as an explicit way of saying "there's nothing here" (otherwise we get an error, since Python expects *something* in the body of a function).

Figure 5 shows the contents of this function after you add the `pass` statement (remember that indentation is essential in Python programs. You can use a few spaces, or a tab, but you must somehow indent `pass`). Once you've entered the function's body, be sure to press *shift-enter* so that Python executes the cell and learns the function you just created.

Once we've set up this do-nothing circuit builder, we call `run_project` to run it and plot the results. I've already written this call and placed it in its own cell, shown in Figure 6. Since we expect to always get back 0, we need only run it once, so I've set `num_shots` to 1. Select this cell (if it's not already selected) and press *shift-enter* to run it.

```
run_project(make_gates_Hello_World, num_qubits=1, num_shots=1)
```

Figure 6: The code to run the circuit for `make_gates_Hello_World` defined in Figure 5

The output should be two pieces of graphics, separated by some text. The output I get on my Mac is shown in Figure 7. Depending on your operating system, the graphics may look different.

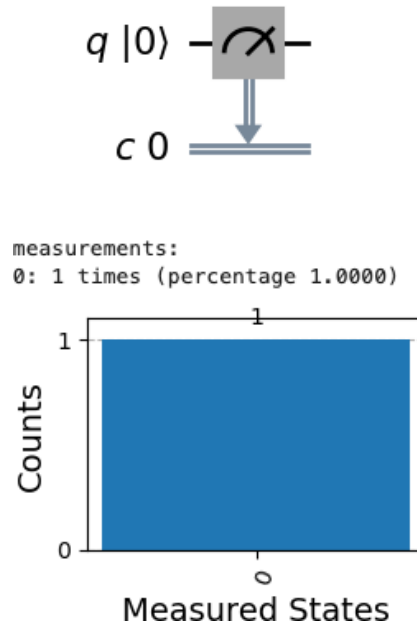


Figure 7: The output I get from our Hello, World program

First we see a diagram showing the circuit itself. There's one quantum bit, labeled q , initialized to $|0\rangle$. Below that is a single classical bit to hold the measurement, called c , initialized to 0. The quantum bit is measured and the result is saved in the classical bit, which is plotted in the bar chart.

Where did that measurement step come from? The helper function `run_project` inserts a measurement at the end of each qubit's processing (here, nothing) so we can see our results. In Qiskit's graphical style, measurement is a little box with a meter inside. The output of the measurement flows down a double-line arrow to a classical bit that is automatically created for us to hold the output of the measurement. Following standard convention, Qiskit draws quantum bits with single lines, and classical bits with double lines.

Below this diagram, `run_project` has provided some text summarizing the measurements. In this case, we measured 0 a single time (since we only

```
def make_gates>Hello_XWorld(qc):
    qc.x(0)
```

Figure 8: Listing for Hello, XWorld

ran the circuit once). Thus the percentage of measurements that were 0 is 1.0, or 100 percent.

Below that is a bar chart graphing this result. Each different output has its own bar, and its height shows how many times that output was measured. Centered at the bottom of each bar is the value it represents. Here we have only one output, so there’s only one bar, and below it is the label 0. At the left is the vertical range for the counts that are displayed. Here that range is from 0 to 1. Finally, at the top of the bar is the count it represents (when the vertical scale is large, it’s often convenient to see the actual count explicitly, rather than estimating it by visually looking at the scale to the left). Here we see that only one state was measured (that is, 0), and it was measured 1 time (since we only ran the circuit once).

If you get these results, then everything is up and running properly!

6.3 Hello, XWorld

Now we’ll make a super-simple quantum circuit with just a single X quantum gate.

The function `make_gates>Hello_XWorld` has one line: `qc.x(0)`, as shown in Figure 8. Let’s pull this apart.

We begin with `qc`, the name of the quantum circuit that `run_program` utility makes for us, which our function receives as an argument. We use this as a prefix when building the gates, telling Qiskit which circuit it should put our gates into.

The gates are placed left to right in the order they’re named in the code. We have only one qubit, so if we create lots of gates they will be appended to that qubit left to right as we read the statements top-down. Here we have only one gate, the X gate, which Qiskit specifies with the lower-case `x`.

Finally, we have to tell Qiskit which quantum bit this new gate should be appended to. We only have one qubit here so naming our intended qubit is kind of superfluous, but circuits can have many qubits. Qubits are numbered with 0 as the first, then 1, and so on. So `qc.x(0)` says, “In circuit `qc`, append

an X gate after any gates that might already exist on qubit 0.”

Now that the circuit has been defined, we call `run_project` to execute it and show the results. The new circuit diagram has a single X qugate immediately after initialization, followed by a measurement. And the measurements, shown in Figure 9, tell us that, as expected, we measured a 1 on our single run. When drawing circuits, Qiskit by default applies colors to different objects (like the gray measurement and blue X gate).

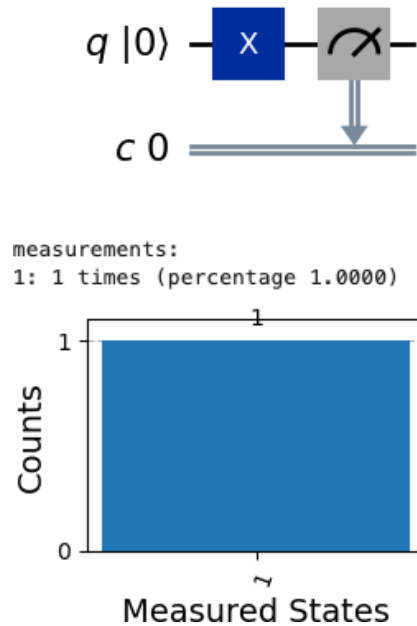


Figure 9: The output from Hello, XWorld

6.4 Hello, HWorld

Now let’s place a single Hadamard, or H gate, on our single qubit. As we know, this will change the qubit from its initial state $|0\rangle$ to the superposition $|+\rangle$, or $(1/\sqrt{2})(|0\rangle + |1\rangle)$.

If we run the circuit many times, we expect about as many 0 measurements as 1 measurements (to simulate the unpredictable nature of real world measurements, Qiskit uses a pseudo-random number generator as part of its measurement process).

```
def make_gates>Hello_HWorld(qc):
    qc.h(0)
```

Figure 10: Listing for Hello, HWorld

The program `make_gates>Hello_HWorld` looks just like the program from the last section, only now we call `qc.h(0)` to tell Qiskit to put an H gate onto qubit 0 in circuit `qc`. The code is shown in Figure 10.

In the cell that calls `run_program` for this circuit I told it to run for 100 shots. The output is a diagram of the circuit and the measured results. As Figure 11 shows, I got 52 measurements of 0, and the remaining 48 were 1. The output plot shows both measured outputs and the number of times they were measured. The Hadamard gate is colored red by default to help this important gate stand out.

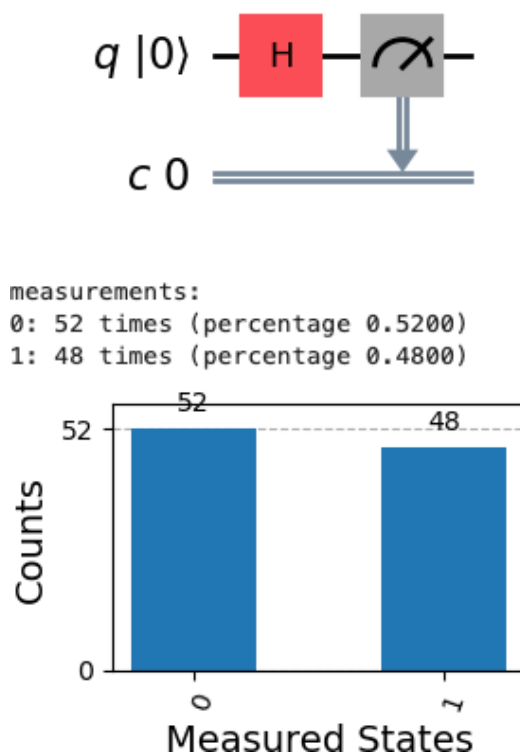


Figure 11: The output from Hello, HWorld

```
def make_gates>Hello_HCXWorld(qc):
    qc.h(0)
    qc.cx(0, 1)
```

Figure 12: Listing for Hello, HCXWorld

Try changing the value of `num_shots` a few times. Remember each time to re-execute the code for `make_gates>Hello_HCXWorld` and then re-execute the call to `run_program` beneath it. The more shots you run, the closer you should get to equal numbers of 0 and 1 measurements.

6.5 Hello, HCXWorld

Finally, let's make a circuit with two qubits.

I'll put an H on qubit 0, and then a CX gate using qubit 0 as the control on qubit 1. This makes a *Bell state*, where the two-qubit system is in an equal superposition of the two states $|00\rangle$ and $|11\rangle$.

The new program, `make_gates>Hello_HCXWorld`, starts as before with `qc.h(0)` putting an H gate on line 0. Then we call `qc.cx(0, 1)` which tells Qiskit to place a CX gate on circuit `qc`, using qubit 0 as the control and qubit 1 as the target. The full code is shown in Figure 12.

Running this program just once (that is, using `num_shots=1`), we expect to get back either 00 or 11.

The more interesting result comes from running it multiple times. For Figure 13 I asked for 100 shots, and you can see I got back roughly the same number of measurements for each state.

Try increasing or reducing the number of shots and see what happens to the measurements.

6.6 Moving On

Our work here is done. We used the `Intro-2026.ipynb` notebook to make sure that our system is up and running, and get our feet wet writing Qiskit code. Now we can start over with a fresh notebook and do quantum teleportation!

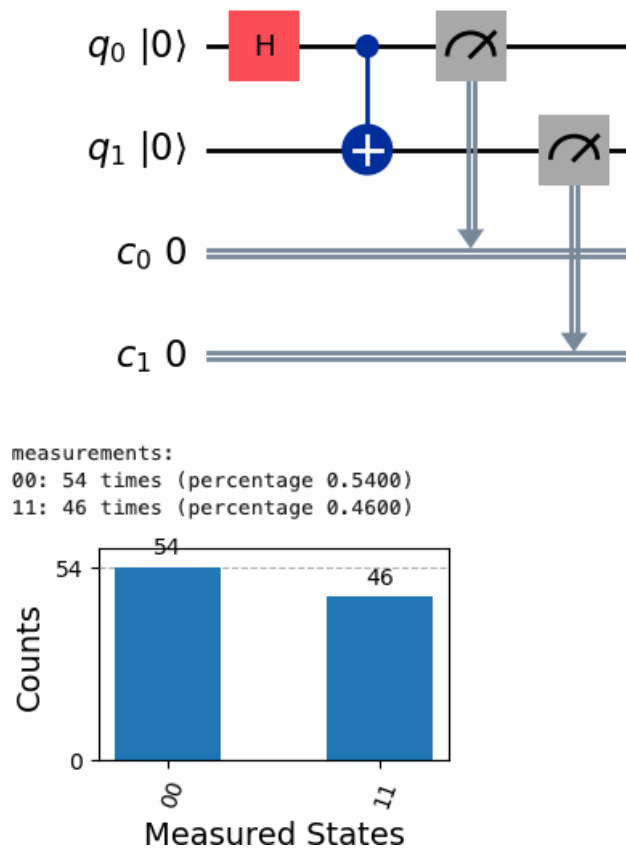


Figure 13: The output of Hello, HCXWorld

7 Quantum Teleportation

The rest of this document will be about writing and running the quantum teleportation algorithm.

When I give this course live, everyone in the room gets paired up with someone else (usually someone sitting next to them). In the first step, everyone is Alice: they pick a secret message and send it to their partner, Bob. In the second step, everyone is Bob, and receives the message sent by their paired Alice.

To keep things clean, I've created one notebook for when you're Alice, and another notebook for when you're Bob.

We're going to use a different simulator for these notebooks than we

used in Section 6. The simulator we used there is designed to support large numbers of runs of complex circuits, so it's not instrumented to let us dig into the details of any specific run.

But here, we will run our circuits only a single time. More importantly, we want to “peek” at the quantum states of the qubits involved and learn their amplitudes.

Why do we need to peek at the quantum amplitudes? Two reasons.

First, it's because everyone is on different computers, running different simulators. In the real protocol, when Alice is done with her step, Bob's qubit b has been placed in a new superposition. But with simulators on different computers, Alice's qubit and Bob's qubit are not the same physical object (in computer terms, they don't share the same memory). We need a way for Bob's qubit to take on the value that Alice computes. I didn't want to depend on fancy operating-system magic for this, so we'll do something simple: Alice will save the state of her qubit in a tiny text file, and then Bob will read that file and initialize his version of the qubit to that value.

To do this, we have to get the amplitudes of Alice's qubit. We can do this here because the simulator is not real quantum hardware. Instead, it's mimicing that hardware numerically. That is, rather than quantum objects actually evolving over time, the simulator is computing explicit numerical variables that it modifies over time, mirroring what happens in real hardware. So we can read back the values of those variables, and thus learn the complex numbers that define a qubit's state.

The second reason for peeking at the state is that it lets us recover the final output from Bob's circuit, and thus receive the secret message that Alice sent. Without this special ability, we'd measure only a 0 or 1 from Bob's output. We could conceivably run the entire protocol many times and estimate Alice's message, but directly accessing the final amplitudes of Bob's qubit lets us get there immediately.

The ability to read the state of a quantum bit is a huge benefit of using a simulator. On real hardware, we can estimate the amplitudes of a qubit with a lot of effort. But with this type of simulator, the numbers are right there for us to use. This is incredibly helpful when writing and debugging quantum algorithms. And, of course, for this project!

A downside of using this simulator is that it doesn't support measurements. This means that we have to implement measurements ourselves, following the quantum theory for how measurements happen. I've written code to do this for you in the helper functions in the Alice and Bob notebooks,

supported by comments there and some additional discussion here.

Let's talk about our overall plan. The process starts with Alice coming up with a secret number. She then encodes that number into the state of her quantum bit s . The secret number might be her birthday, or phone number, or any number that she chooses. It gets represented as a number between 0 and 1. For example, if her number was 3141, she'd save it as 0.3141. She'll create a quantum bit where this is the amplitude on the state $|0\rangle$. In other words, she'll create a quantum bit with state $\alpha|0\rangle + \beta|1\rangle$, where α has the real value 0.3141. The corresponding value of β is now forced by the physical requirement that $|\alpha|^2 + |\beta|^2 = 1$, so my code computes this β for you.

Alice will then run her part of the teleportation circuit. Ultimately, her computation will use entanglement to change the state of the qubit b that Bob's holding.

Alice will share three pieces of information with Bob: the states of the two classical bits that she's measured, and the name of a text file that holds the amplitudes of Bob's qubit b .

Bob will use this information to initialize and run his circuit, producing a final value for b . Bob then uses the simulator to "peek" at the final coefficients of b . In our example, he'll find that the amplitude of state $|0\rangle$ is 0.3141, thereby receiving Alice's message!

So that's the big plan. To look at it in some more detail, I'll start by working through Alice's notebook, and then Bob's.

The code is documented throughout, but the comments are usually short. The purpose of these notes is to expand on those comments and offer some supporting information.

The first chunk of each notebook is all about support, or infrastructure. It creates functions to build and initialize the quantum circuits, set up and run the simulator, and interpret its results. None of this is directly related to quantum teleportation, but it's how we work with the Qiskit library and its simulators.

If you're not interested in these mechanics right now, you can skip their discussions here, and when running the notebooks, you can just execute those cells without paying attention to their contents. You can always come back later and look at the code and read the discussions

Do read the second part of the notebooks and their commentary here, though, because that's where you're actually doing quantum teleportation!

For reference, the typical circuit diagram for quantum teleportation is repeated here in Figure 14. Remember that we won't be explicitly putting

any measurement steps into our circuits, because we'll be performing all measurements ourselves.

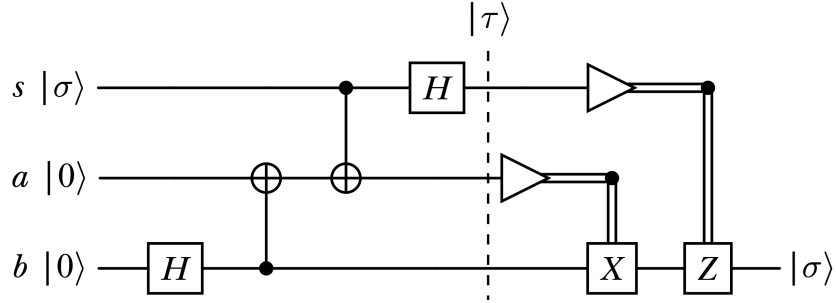


Figure 14: teleport-circuit XX

8 Notebook Teleport-Alice-2026

This is the notebook that you'll use when you're playing the role of Alice: you select a secret number, encode it into the state of a qubit, and then teleport that state onto an entangled qubit held by Bob.

Section 1 of this notebook imports the libraries we'll need, and defines three functions that we'll call to manage the simulation.

Section 2 holds the code you'll interact with. This is where you set up your secret message, and then call the functions in the first section to build, run, and measure the quantum circuit. It will then save the state of the entangled qubit to a file, and tell you the classical measurements of the other two qubits.

Figure 15 shows a block diagram of how we use the support programs to implement Alice's half of the protocol.

8.1 Section 1: Alice's Infrastructure

This section imports the libraries we'll use, and then defines functions to handle Alice's part of the teleportation algorithm.

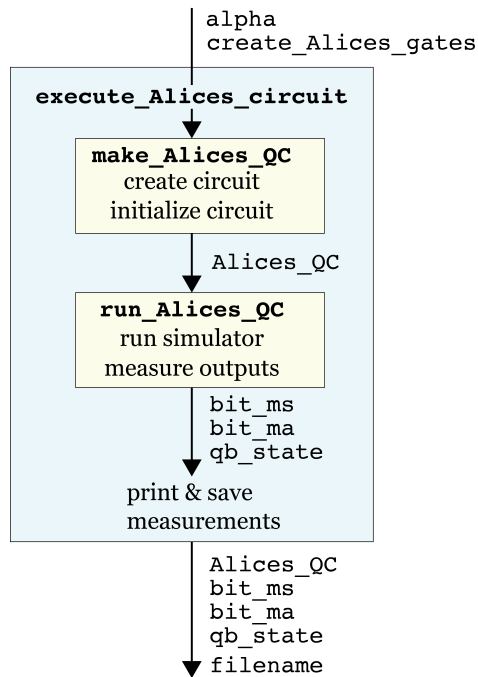


Figure 15: An overview of Alice's code. Function names are shown in bold, inputs and outputs in fixed-width type

8.1.1 Imports

We start off by importing the libraries (and pieces of libraries) that we'll be using.

QuantumCircuit: Like before, the object that holds the circuit itself.

QuantumRegister: An object that holds one or more quantum bits.

Statevector: This is our new simulator (from the `qiskit.quantum_info` part of the library). Its name comes from its ability to tell us the vector, or list of numbers, that describe the state of a qubit.

I'm using **QuantumRegister** rather than building the circuit directly with **QuantumCircuit** as we did in Section 6 because this approach lets us give symbolic names to our qubits, which I think is useful here.

8.1.2 `make_Alices_QC`

Alice's first step is to build her part of the quantum circuit. That's what this function does.

`make_Alices_QC` takes two arguments: Alice's secret message (in `alpha`), and a callback function that defines the quantum gates that make up Alice's circuit. It returns the fully assembled and initialized circuit, ready to be run.

The reason I'm using a callback to define the gates is so that you can get practice with writing that code. Most of the infrastructure is one-time work, or stuff that you'll copy and paste and modify if you continue and do some more quantum programming. But the circuits are the things that you design and test and debug and really focus on. So I've put the circuit definition into its own routine (defined later) where you can clearly see the circuit you've created, away from all the support stuff that stays relatively constant from one project to the next.

The first chunk of code in `make_Alices_QC` starts with the real number `alpha` between 0 and 1 that makes up Alice's message, and turns that into a state $|\sigma\rangle = \alpha|0\rangle + \beta|1\rangle$ for the quantum bit s . The mechanism is that we place the message directly into α , so that's just a real number with no complex part. To compute β we use the requirement that all quantum states obey $|\alpha|^2 + |\beta|^2 = 1$ for complex numbers α and β . Since α is real, we can compute β from $\beta = \sqrt{1 - |\alpha|^2}$ (the absolute value signs around α are overkill in this situation, but they're conventional and they don't hurt). I make a two-element list called `sigma_state` that holds these values.

The next chunk creates three one-bit `QuantumRegister` variables for qubits s , a , and b . The advantage of using `QuantumRegister` in this way is that we can attach a short string, or name, to each qubit. These names will show up when we use library functions to draw our circuit diagrams.

We then instantiate the circuit by calling `QuantumCircuit` with these three registers. We initialize qubit s with the quantum state `sigma_state` that we just made. Now that the circuit is set up and initialized, we call the callback that populates the circuit with quantum gates.

That completes the building phase for Alice's circuit, so we return it.

8.1.3 `run_Alices_QC`

Once Alice has created her circuit, she will want to run it. That's the job of this next function, `run_Alices_QC`.

We begin by running the simulator. The syntax is to call the somewhat lengthy `Statevector.from_instruction(Alices_QC)`. This tells Qiskit to use the `Statevector` simulator, and run it using the “instruction” (or circuit diagram) that Alice just made.

As I mentioned earlier, the `Statevector` simulator is the right one for this project because it lets us directly access the coefficients on the state of any qubit, something we can never do in nature. Unfortunately, this simulator doesn’t provide built-in mechanisms for measuring quantum bits. Since we do need to measure two of Alice’s qubits, the next chunk of code performs a manual implementation of how quantum measurement happens in practice.

Alice’s circuit has three quantum bits, so the output of calling the simulator for a 3-qubit system is a superposition of up to $2^3 = 8$ quantum states, each described by a pair of complex numbers, or amplitudes. The squares of these amplitudes tell us the probability that when we measure that qubit we’ll get back a classical 0 or 1.

Before we continue, I need to address some book-keeping.

Let’s say that you’ve been doing computer graphics with a right-handed coordinate system for a long time, and then someone hands you a model built in a left-handed system. You could try to work with it as-is and constantly remind yourself to write unfamiliar code to manage its coordinates, and then of course you’ll forget somewhere, and then you get to spend some quality time debugging your code. You could save yourself all the hassle by converting the model to a right-handed system at the start.

That’s what I do next. The analogy to the right-versus-left coordinate systems is how we number the states in a superposition. In our case, we have eight pairs of complex amplitudes that are naturally kept in a Python list, or array. So we need to agree on which state appears in each position in the list.

When we make a measurement of a three-qubit system, we get back one of eight bitstrings: 000, 001, 010, and so on. Since binary 000 is decimal 0, the amplitudes corresponding to measuring 000 appear at index 0 in the list, the amplitudes corresponding to measuring 001 appear at index 1 in the list, and so on.

All well and good, except for one sticking point: does the bitstring 100 describe the state where we’re considering the qubits top-down in the diagram, or bottom-up? Either way is fine, and both are used in practice. But we have to pick one way and stick with it. Otherwise, we won’t know if the probabilities for state 100 refer to having the 1 in the top-most or

bottom-most position.

I've assigned my bitstrings top-down for years, and it's now pretty much automatic for me to think that way. So when I think of state 001, I think of the two upper qubits as 0 and the bottom qubit as 1.

Unfortunately, Qubit assigns the bits in a bitstring to the diagram's qubits bottom-up. The upshot is that the amplitudes for what I think of as bitstring 001 end up in index 4 of Qubit's list, rather than index 1. Similarly, 011 is in index 6 rather than 3, 100 is in index 1 rather than 4, and the amplitudes for 110 are in index 3 rather than 6.

Just as someone used to right-handed models would convert a left-handed model as soon as they received it, I convert Qiskit's sequence of outputs from bottom-up to top-down as soon as the simulator returns the array that holds them. It's just one line of code that shuffles the entries around. I've gone into all of this because I think it would be plenty mysterious without discussion! If you're happy using bottom-up naming, you could skip this shuffling, but you'd have to adjust the indexing of the following steps as well.

Now we're ready to simulate measurements. To make measurements we need to work with probabilities, so I square the amplitudes to convert them to probabilities.

Now we're going to simulate a measurement on the top two qubits. There are only four possible outcomes for these two qubits: 00, 01, 10, and 11. But the simulator is returning the probabilities for all 8 states in the supposition of Alice's 3 qubits. We don't care about the state of the third qubit. If we measured that qubit, it could only result in 0 or 1. So the probability of measuring 00 on the first two qubits is the summed probability of measuring 000 or 001 (remember that we're assigning bits left-right as we move top down, so since Bob's qubit b is at the bottom of the diagram, it comes at the end of the bitstring). The probability of measuring 01 on the first two qubits is the summed probability of measuring 010 or 011, and so on for 10 and 11.

Once I've computed these four probabilities, we need to pick one state to emerge as the measured result. Quantum theory tells us that the probabilities themselves guide which state will result from a measurement. Happily, the `numpy` library offers the function `numpy.random.choice` which does just that. The next few lines take the decimal result from that call, turn it into a 2-digit binary number, and assigns the first bit to the variable `bit_ms` and the second bit to `bit_ma`. These variables are the results of our simulated measurement for qubits s and a respectively.

The last thing to do is to get the value of Bob’s qubit b from the simulator on Alice’s computer over to the simulator on Bob’s computer. In the real world, the act of measuring Alice’s qubits would, thanks to entanglement, immediately change the state of b (no matter how far away it is). But because Alice and Bob are using distinct simulators, we need to somehow get the amplitudes of Bob’s qubit out of this simulator’s memory and save them, so we can give them to Bob later.

This is impossible to do directly in the real world. The best we can do is invest a whole lot of effort to get approximations of the qubit’s amplitudes.

But we’re using a simulator! And not just any simulator, but one that explicitly represents the state of qubits by their amplitudes. So we can do something we can’t do in reality, and simply read the amplitudes of the state of b out of the simulator’s memory. These aren’t perfect values (they’re limited by the floating-point precision of the simulator), but for our purposes they’re way more than accurate enough.

We just need to pluck the amplitudes of the correct state out of the list given to us by the simulator. The measurement we just did tells us which set of amplitudes to choose. Suppose that Alice’s measurement of her two qubits were 0 and 1, or the bitstring 01. So the amplitude on state $|0\rangle$ for b must be the amplitude of system state 010, and the amplitude on state $|1\rangle$ for b must be the amplitude of system state 011. I save those two amplitudes in a two-element list.

Because floating-point calculations can make numbers drift a little, I normalize them to enforce the requirement that $|\alpha|^2 + |\beta|^2 = 1$ (this is probably not necessary, but I chose to be cautious).

Alice’s circuit has been executed, we’ve measured the first two qubits, and we know the state of qubit b . We’re done, so we return those three results.

8.1.4 `execute_Alices_circuit`

The third function in Alice’s support section is a convenience function that runs the two functions we just saw, and saves the state of b to a text file.

Its inputs are `alpha`, Alice’s secret message as a number from 0 to 1, and `create_gates`, Alice’s callback that places her quantum gates into a blank circuit.

I call `make_Alices_QC` to build her circuit, and `run_Alices_QC` to run it. I make a filename from a pseudo-random number and save the state of b into that file. I use `np.savetxt` to write the file because it saves complex

```
def create_Alices_gates(Alices_QC, qs, qa, qb):
    Alices_QC.h(qb)      # Bob's H
    Alices_QC.cx(qb, qa) # Bob to Alice CX
    Alices_QC.cx(qs, qa) # Signal to Alice CX
    Alices_QC.h(qs)      # Signal's H
```

Figure 16: Listing for `create_Alices_gates`

numbers in a way that its corresponding read routine is able to extract.

I return the circuit, the measurements, and the state of qubit *b*.

8.2 Section 2: Alice's Step

This is the part where you, as Alice, actually drive the show. You have three cells to execute.

8.2.1 Alice's Step 1: Populate the Circuit

Alice needs to provide a callback, which I've called `create_Alices_gates`, that will populate an empty but initialized quantum circuit with her quantum gates. It takes as input Alice's circuit `Alices_QC`, and variables representing the three qubits `qs`, `qa`, and `qb`. The listing is shown in Figure 16 (the `#` sign and everything after are comments, which you can include or not as you please).

In the code, you can see why I used `QuantumRegister` when I built the circuits: it let us name the qubits symbolically by saving them in variables with meaningful names. So Alice can place an *H* gate on Bob's quantum bit, which we named `qb`, by writing `Alices_QC.h(qb)`. Otherwise we'd have to use a numerical index into an array of quantum bits. I think using a name is a whole lot clearer.

Alice then inserts a *CX* using *b* as a control on *a*, a *CX* using *s* as a control on *a*, and finally inserts an *H* onto *s*.

8.2.2 Alice's Step 2: The Secret Number

Here's where Alice declares her secret number and saves it as `alpha`.

Pick an integer, then put a zero and decimal point in front of it to create a number between 0 and 1. For example, if you choose the secret number

271828 you'd save it as 0.271828. In practice, long secret numbers (that is, with lots of digits) invite floating-point noise that could give us a wrong result. I've never seen 8 digits fail, but if things don't work for you, try using a secret with fewer digits.

8.2.3 Alice Step 3: Execute the Circuit

Alice hands her secret string and callback routine to `execute_Alices_circuit`, which builds and runs the quantum circuit, simulates the measurements at the end, and saves the quantum state of b into a file.

That's it; Alice is done!

Typical output is shown in Figure 17.

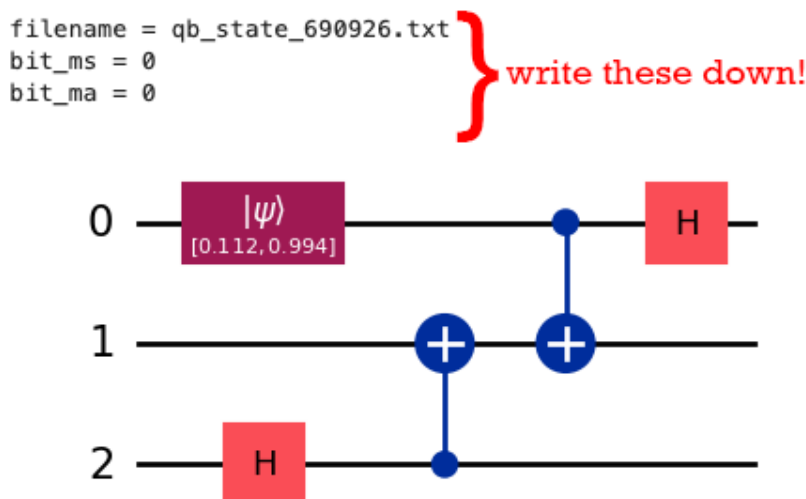


Figure 17: A typical instance of Alice's final output (your numbers will almost surely be different). At the top is the information she needs to give Bob: the values of the two measured bits (s and a), and the name of the file holding the state of b . I added a reminder that you need to write down the filename and bit values.

The most important part of this output are the lines at the top, where the program prints out the two classical bits that Alice needs to tell Bob, along with the name of the file holding the state of Bob's qubit. Below that text there's a drawing of the circuit, so we can confirm what we did (note that there are no icons representing measurement, since we did that ourselves).

Important! Write down the measurements and the file name!
This is the information you need to tell to Bob to finish the protocol.

8.3 Re-running the Notebook

If you want to close the notebook, *make sure you first record Alice's measurements and the file name!* These are printed in the final output cell, just above the circuit diagram, as shown in the upper left of Figure 17. Once the notebook is closed, you won't be able to get these back, and you'll have to run the whole thing again (producing new measurements and a new file name).

If you do accidentally close the notebook before recording these things, you can run it again efficiently. Open a fresh version of the notebook and scroll down so you can set `alpha` to your chosen secret number. You can just type in the number you want; there's no need to execute the cell with *shift-return* because we'll hands that next. Go to the *Run* menu at the top of Jupyter's window and select **Run All Cells**. This does just what it says: it executes every cell in the notebook in order from top to bottom.

This time, remember to write down the measurements and file name before closing the notebook!

9 Notebook Teleport-Bob-2026

It's time for Bob to open his notebook and finish the protocol. Bob needs to talk to Alice and get her two measured bits, and the name of the file she wrote.

Bob's notebook assumes that the file is in the same folder as the notebook itself. If you're being both Alice and Bob, then you don't need to do anything, as Alice's output will be in that folder. If Alice and Bob are on different computers, then Bob needs to somehow transfer, copy, or move the file that Alice wrote into his directory so that he can read it.

The structure of Bob's notebook is very much like that of Alice's: an infrastructure section with import statements and three support functions, and a section that Bob uses to call these functions and run the circuit.

9.1 Bob's Infrastructure

Bob's support functions largely parallel Alice's, and have the same names except with Bob in place of Alice.

An overview of these functions and their inputs and outputs is shown in Figure 18.

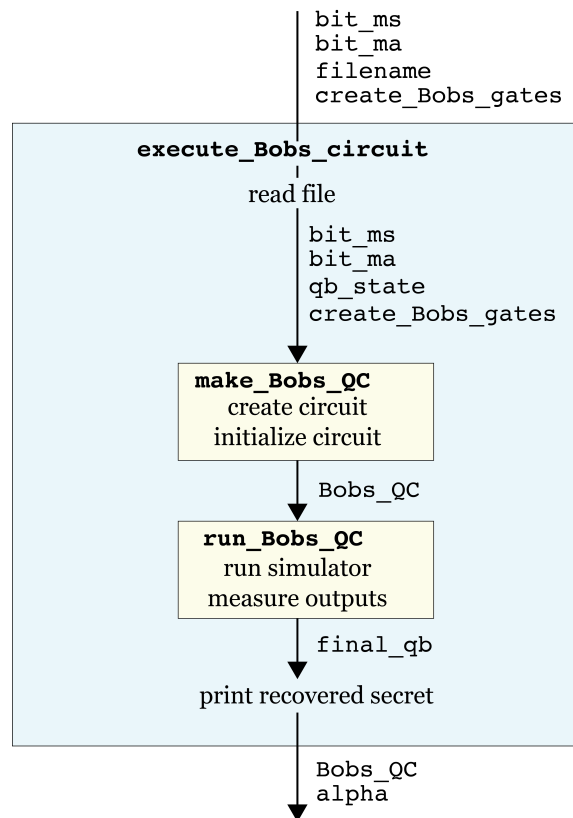


Figure 18: An overview of Bob's support code. Function names are shown in bold, inputs and outputs in fixed-width type

9.1.1 Imports

Bob's import cell is identical to Alice's, so you can refer to that description for what each library is about.

9.1.2 `make_Bobs_QC`

This routine takes four arguments: the classical bits `bit_ms` and `bit_ma` that Alice measured, the initial state of quantum bit b in the variable `qb_state`, and a callback routine to populate Bob's circuit with gates.

We begin by creating a circuit with three qubits, named s , a , and b .

But wait a second - why three qubits? According to Figure 14, Alice has measured her two qubits, so they don't matter any more. Now we have just one quantum bit, and two classical bits.

The reason is that we somehow have to provide control signals to the CX and CZ gates in Bob's circuit. In Qiskit, there's no way to use a classical bit to control a quantum gate (at least, no way that I know of). So I make quantum bits for the control lines, and I initialize each one to either $|0\rangle$ or $|1\rangle$ based on the bits Alice reported to us.

Then I initialize the quantum bit b using `qb_state` from the saved text file.

Finally, I call the callback routine to fill up the empty circuit with gates.

Now that the circuit `Bobs_QC` is ready to run, I return it.

9.1.3 `run_Bobs_QC`

This routine takes one argument: the circuit `Bobs_QC` made by the previous function. It then runs this circuit and reports the final amplitudes on qubit b . Since this should be the teleported version of Alice's qubit a , whose amplitude on $|0\rangle$ was her secret (real) number, the amplitude on $|0\rangle$ of this qubit should be that same number.

To get things rolling, I call `Statevector.from_instruction` to evaluate the circuit we built.

Then I shuffle the amplitudes around like as I did in `run_Alices_QC`, again so that bitstrings can be read top-to-bottom, rather than the bottom-to-top order used by Qiskit. I square the amplitudes to get their corresponding probabilities.

The final step is to work out the value of b . If we measure all three qubits and get back the bitstring 000, 010, 100, or 110, then we know that b was 0. So I just add the probabilities of those four states together to get the probability we'd measure a 0 on b , knowing that this probability is the square of the amplitude that Alice started with. For completeness (though we don't need it), I also add together the four probabilities that could give

us a 1 for b .

Given the probabilities of measuring 0 or 1, I take their square roots to get their corresponding amplitudes.

That's it, and I return the amplitudes.

9.1.4 `execute_Bobs_circuit`

This function is a convenience to run the previous two in sequence. It takes four arguments: `bit_ms` and `bit_ma` corresponding to Alice's measurements, the filename that holds Alice's computed state of b , and the callback function to populate Bob's circuit.

I then use `np.loadtxt` to read the file Alice's code wrote, giving us the state of qubit b at the end of Alice's computation.

I build Bob's circuit, which I name `Bobs_QC`, and run it, getting back the amplitudes on b . I then cast `alpha`, the amplitude of $|0\rangle$ on b , into a real number so that we don't have to see or deal with the imaginary component (which should be 0 anyway).

The function ends by returning Bob's circuit, and the value of `alpha` that we extracted at the end.

9.2 Bob's Step

Now it's time for Bob to retrieve the secret message that Alice has teleported to him.

If you were Alice in the last step, then you need to tell the person playing your recipient Bob what your classical bits were, and the name of your file.

If you're doing this lab with two people, each can be Alice, and now each can be Bob, receiving the other's message. In a bigger group, Alices and Bobs can pair up in any way you like.

The key thing is that you, Bob, know the bits that Alice measured, and the name of the file holding the amplitudes of qubit b at the end of her computation.

9.2.1 Bob Step 1: Populate the Circuit

As before, Bob needs to write a callback routine to populate his circuit. It's just two gates that he can refer to symbolically: one a CX gate (created with the `cx` function), and the other a CZ gate (created with the `cz` function).

```
def create_Bobs_gates(Bobs_QC, qs, qa, qb):
    Bobs_QC.cx(qa, qb) # CX
    Bobs_QC.cz(qs, qb) # CZ
```

Figure 19: Listing for `create_Bobs_gates`

This routine is called `create_Bobs_gates`, and it takes the same parameters as Alice’s version. The full listing is shown in Figure 19. Again, you can leave out the comments if you like.

9.2.2 Bob Step 2: Enter Alice’s Result

Here we just save the three variables holding the information Alice has told us. The variable `bit_ms` holds the value (0 or 1) that Alice measured for her s qubit, the variable `bit_ma` holds the same information for Alice’s a qubit, and `filename` holds the randomized file name that Alice wrote, holding the initial state of Bob’s qubit b .

9.2.3 Bob Step 3: Execute the Circuit

Bob hands Alice’s data and his callback to `execute_Bobs_circuit`, and it returns with Bob’s circuit and the value of `alpha`, the amplitude of $|0\rangle$ on Bob’s qubit. A typical output is shown in Figure 20.

If all has gone well, the value printed out by Bob’s final step will match the value that Alice chose.

Secret message received via quantum teleportation!

10 Hooray!

Congratulations, you’ve implemented and simulated quantum teleportation!

It’s definitely time to celebrate! Do something nice for yourself, you quantum programmer, you.

If you’re at the live lab, come on up and get one of the quantum computing merit badge stickers I’ve made for you! If you’re at home, print one out from Figure 21.

received alpha = 0.112358

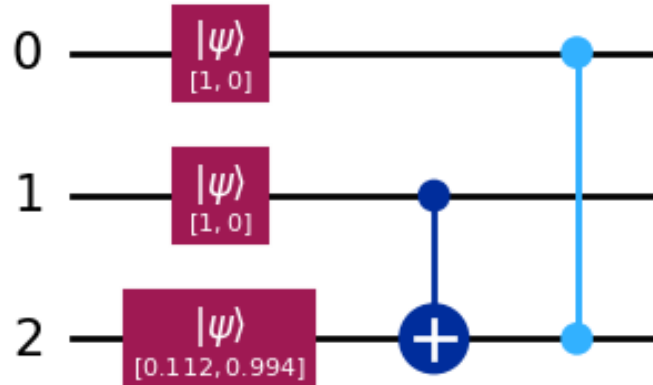


Figure 20: Bob’s output, giving us the recovered value of `alpha`, and the circuit diagram. Your numbers will almost surely be different.

References

- [Glassner(2025)] Andrew Glassner. 2025. *Quantum Computing: From Concepts to Code*, No Starch Press, 2025
- [Qiskit(2026)] Qiskit documentation, 2026. <https://qiskit.qotlabs.org/docs/guides/tools-intro>

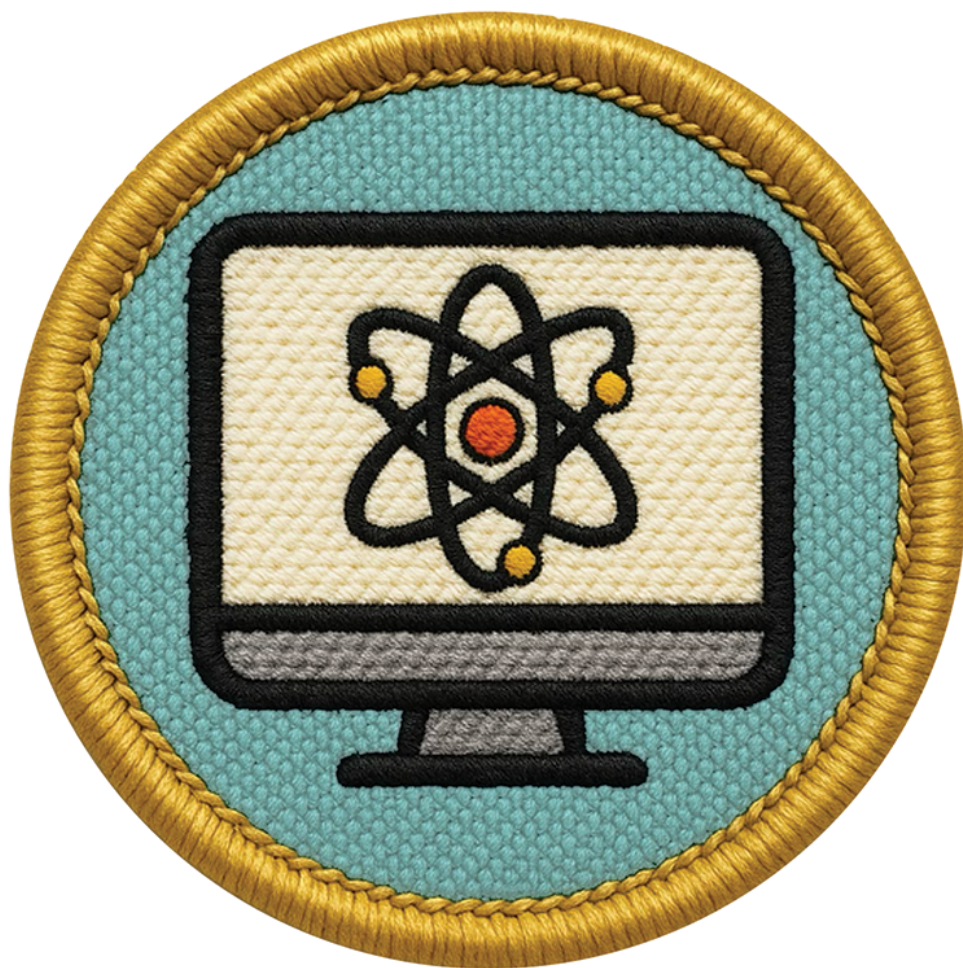


Figure 21: A quantum programming merit badge sticker I designed and hand out to lab attendees. If you're following along at home, feel free to print it out!

QUANTUM COMPUTING

From Concepts to Code

by Andrew Glassner



**no starch
press®**

San Francisco

QUANTUM COMPUTING. Copyright © 2025 by Andrew Glassner.

All rights reserved. No part of this work may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage or retrieval system, without the prior written permission of the copyright owner and the publisher.

Printed in the United States of America

First printing

29 28 27 26 25 1 2 3 4 5

ISBN-13: 978-1-7185-0400-4 (print)

ISBN-13: 978-1-7185-0401-1 (ebook)



Published by No Starch Press®, Inc.
245 8th Street, San Francisco, CA 94103
phone: +1.415.863.9900
www.nostarch.com; info@nostarch.com

Publisher: William Pollock

Managing Editor: Jill Franklin

Production Manager: Sabrina Plomitallo-González

Production Editor: Miles Bond

Developmental Editors: Annie Choi and Suzanne Olivier

Cover Illustrator: Josh Kemble

Interior Design: Octopod Studios

Technical Reviewer: Ronald T. Kneusel

Copyeditor: Rachel Head

Proofreader: Daniel Wolff

Library of Congress Control Number: 2025010328

For customer service inquiries, please contact info@nostarch.com. For information on distribution, bulk sales, corporate sales, or translations: sales@nostarch.com. For permission to translate this work: rights@nostarch.com. To report counterfeit copies or piracy: counterfeit@nostarch.com. The authorized representative in the EU for product safety and compliance is EU Compliance Partner, Pärnu mnt. 139b-14, 11317 Tallinn, Estonia, hello@eucompliancepartner.com, +3375690241.

No Starch Press and the No Starch Press iron logo are registered trademarks of No Starch Press, Inc. Other product and company names mentioned herein may be the trademarks of their respective owners. Rather than use a trademark symbol with every occurrence of a trademarked name, we are using the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The information in this book is distributed on an “As Is” basis, without warranty. While every precaution has been taken in the preparation of this work, neither the author nor No Starch Press, Inc. shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in it.

7

TELEPORTATION

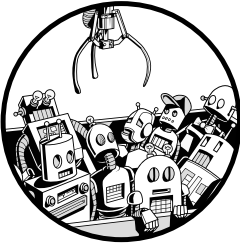
Hey, hey, hey.

You know, don't be mean.

We don't have to be mean.

*'Cuz remember, no matter where you go . . .
there you are.*

—Earl Mac Rauch, *The Adventures of Buckaroo Banzai Across the 8th Dimension*,
1984 [170]



In this chapter, we'll meet our first complete quantum algorithm! This algorithm doesn't perform a calculation or give us the answer to a specific problem. Instead, it performs a unique, fascinating task: moving the quantum state of one qubit to another qubit, located anywhere in the universe. And it does this *instantaneously*.

Wait, the no-cloning theorem from Chapter 5 tells us we can't do this, right? The theorem does say that we can't make a copy of a quantum state, but it doesn't prohibit us from *moving* a state from one qubit to another, leaving the original qubit in a different state from how it started.

This process has an exciting name: *quantum teleportation*, or just *teleportation*. It's not quite teleportation the way the term is used in science fiction like *Star Trek*, though. We really do communicate the states of a quantum bit from *here* to *there*, but there are four big differences.

The first is that we're not transferring any kind of matter. We're only communicating the state of a qubit. Even if we transferred the state of enormous numbers of qubits, we still don't have any means for assembling the physical objects that have been put into those states into a grumpy but humane doctor, an exploding warp drive, or even a rock.

The second difference is that we can't send the description of the qubit anywhere we like. We can only transfer the state of a qubit to another qubit it's already been entangled with, and which is already present at the receiving site.

The third difference is that to reliably transfer the state of a quantum bit from one place to another, we must also exchange two classical bits over normal, classical channels, such as radio. That means we can't rely on this method to share information unless we also share some classical bits over conventional channels.

Finally, the fourth difference is that when we move a quantum state from one qubit to another, the state of the original qubit is changed, and we can't recover its original value.

Given all of these qualifiers, it might be better to call this quantum state *transfer* rather than teleportation. It's not *Star Trek* by a long shot, but transferring the state of one quantum particle to another quantum particle is still pretty cool.

Three features that real quantum teleportation has over the fictional version are that *distance doesn't matter*, *nothing can interfere with the process*, and *the original must be destroyed to be transported*. The first property means that the source and target can be literally anywhere in the universe. The second two properties protect us from ever accidentally creating an "evil Spock" [176].

In this chapter, we'll work a lot with explicit qubit states and the matrices of the operators that modify them. This is unusual. Most of the time, when we analyze a quantum algorithm, we work entirely (or nearly so) with algebra and rarely get down to the level of coefficients. Most of the rest of this book follows that approach. But sometimes working with the actual coefficients can be illuminating, bringing us a little closer to the mechanics of quantum computing. It also allows us to view an algebraic result in a different way, if there aren't too many qubits involved. For these reasons, in this chapter we'll spend most of our time with components, and we'll see explicitly how the operator matrices manipulate the elements of the ket matrices. You won't need to memorize any of these eight-by-eight matrices, as they're all built up from the smaller two-by-two matrices that we're already familiar with.

Okay, enough prep. Let's get teleporting!

The Teleportation Thought Experiment

A great way to think about teleportation is in terms of a story, or what physicists call a *thought experiment*. This story involves two characters. In physics thought experiments with two characters, they are almost always named Alice and Bob, so I'll carry on that tradition here.

In this story, we imagine that Alice and Bob are separated by a great distance: Maybe Alice is on Earth, and Bob is on Mars. Alice has run some algorithm that produces a quantum state, which I'll call the *signal*, described by a ket s with the state $|\sigma\rangle = \alpha|0\rangle + \beta|1\rangle$, where as always $|\alpha|^2 + |\beta|^2 = 1$. Producing this state is only the first part of a two-part computation. Bob is ready to take over from here and finish the computation, so he needs to have a qubit in the state $|\sigma\rangle$.

Alice could send her physical qubit to Bob, so he can work with it. But let's say that Bob is so remote, and sending things is so slow and expensive, that there's no practical way for Alice to physically send her qubit to Bob.

To get around this limitation, suppose that Bob has taken a qubit named b to Mars. Taken together, Alice's qubit s and Bob's qubit b form a two-qubit system $s \otimes b$. When we think of s and b as a system, it doesn't matter that the qubits are far apart from one another.

Because s is in the state $|\sigma\rangle$, it would be great if there were some sequence of operations that Alice, or Bob, or both of them could follow that would give them the qubit system state $|\sigma\rangle \otimes |\sigma\rangle$. Then Bob's qubit b would also be in the state $|\sigma\rangle$, and they'd have teleported the signal! Unfortunately, this means making a copy of $|\sigma\rangle$, and we know that the no-cloning theorem prohibits that.

Maybe we can avoid cloning if Alice's qubit is changed during teleportation. For example, it might go from $|\sigma\rangle$ to some other state, $|\omega\rangle$. Now if we can put $|\sigma\rangle$ onto Bob's qubit, the new system will be $|\omega\rangle \otimes |\sigma\rangle$. There will be no cloning, and they'll have teleported the signal!

That would be great, but nobody has found a way to do it.

A way that does work requires giving Alice one more qubit. This extra qubit (let's call it a , for auxiliary) will help us perform teleportation.

But how does this help Alice, in her lab on Earth, modify Bob's qubit on Mars? The answer is to link all three qubits together, so that when Alice manipulates her qubits s and a , those operations have an effect on Bob's qubit b .

We know how to do that: Use entanglement! If Alice and Bob created an entangled pair before Bob left, and each kept one qubit of the pair with them, operations on either qubit could affect the other.

We still need some way for Alice to move $|\sigma\rangle$ onto Bob's qubit. The key idea is to create a system state of three qubits that I call the *teleportation state*, which I'll write as $|\tau\rangle$. This is the heart of the whole algorithm. Figure 7-1 shows the teleportation algorithm in two steps.

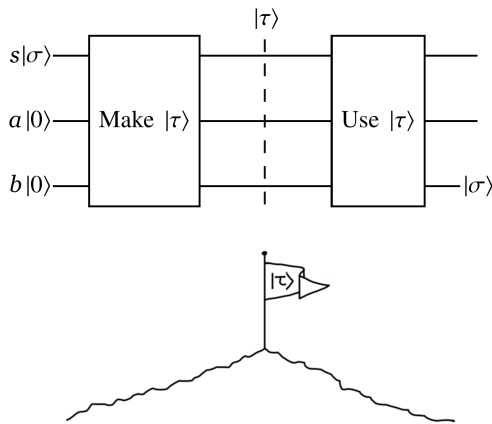


Figure 7-1: The teleportation algorithm can be viewed as two steps.

We can view the first step as climbing a hill to put the qubits into the teleportation state $|\tau\rangle$. We plant our flag (the state $|\tau\rangle$) at the top of the hill and then head down the other side, using $|\tau\rangle$ to put Bob's qubit b into the state $|\sigma\rangle$ that Alice's qubit s was initially in. We don't care about the final states of qubits s and a , so I've left them blank in the figure.

Because the teleportation state $|\tau\rangle$ is at the center of the whole process, let's take a closer look at it.

The Teleportation State $|\tau\rangle$

Let's write the qubits s , a , and b in the teleportation state. A 3-qubit state has $2^3 = 8$ elements. Creating the teleportation state takes only a few quantum gates, and I'll show you that circuit later in this chapter. For now, I'll ask you to take it on faith that Alice and Bob can create $|\tau\rangle$.

The teleportation state involves the qubits s , a , and b . Let's write these in order from top to bottom, as in Figure 7-2. We'll see that the three qubits are entangled together, so their output is the single entangled state $|\tau\rangle$.

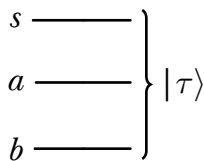


Figure 7-2: The three qubits s , a , and b arranged from top to bottom, making up the entangled state $|\tau\rangle$

The teleportation state $|\tau\rangle$ is an equal superposition of four states. Each is the original $|\sigma\rangle$, perhaps transformed by one or two specific operators.

The state $|\tau\rangle$ is shown in Equation 7.1.

$$|\tau\rangle = \frac{1}{2} \left(|00\rangle I |\sigma\rangle + |01\rangle X |\sigma\rangle + |10\rangle Z |\sigma\rangle + |11\rangle XZ |\sigma\rangle \right) \quad (7.1)$$

At first glance, it looks like we've cloned $|\sigma\rangle$ not just once but three times. But a closer look reveals that there's been no cloning. What we've done is create additional states involving α and β in a single superposition.

We've been doing this kind of thing for several chapters now. For example, suppose we apply an H qugate to $|\sigma\rangle$. The resulting state is shown on the right side of Equation 7.2 (recall our convention that $\vee = 1/\sqrt{2}$).

$$H |\sigma\rangle = \vee \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \vee \begin{bmatrix} \alpha + \beta \\ \alpha - \beta \end{bmatrix} \quad (7.2)$$

The final state is an equal superposition of $(\alpha + \beta) |0\rangle$ and $(\alpha - \beta) |1\rangle$. Both coefficients α and β appear twice, but we haven't cloned anything. Eventually, we'll make a measurement, causing this superposition to collapse, and only one of the states will be associated with the qubit. The same is true of $|\tau\rangle$.

Let's return to $|\tau\rangle$ in Equation 7.1. It's composed of four states in an equal superposition.

In the first state, $|00\rangle I |\sigma\rangle$, the $|00\rangle$ term refers to the two-qubit state $|0\rangle \otimes |0\rangle$. We then tensor this with $I |\sigma\rangle$. This is really just $|\sigma\rangle$, but I included the I for consistency with the other states.

The second state, $|01\rangle X |\sigma\rangle$, tells us to first form $|0\rangle \otimes |1\rangle$ and then tensor that with $X |\sigma\rangle$, or the result of applying the X qugate to the original $|\sigma\rangle$.

The third state, $|10\rangle Z |\sigma\rangle$, is like the previous one. We first form the state $|1\rangle \otimes |0\rangle$ and then tensor that with $Z |\sigma\rangle$.

Finally, $|11\rangle XZ |\sigma\rangle$ tensors together $|1\rangle \otimes |1\rangle$ with the state made by applying Z and then X (in that order) to $|\sigma\rangle$ (remember that we read algebraic operators from right to left).

It's the structure of $|\tau\rangle$ that enables teleportation. In Figure 7-2, the first two qubits of $|\tau\rangle$ correspond to s and a and the third to b .

If Alice measures qubits s and a , then as usual she'll get back a single bit for each. Let's say she finds $s = 1$ and $a = 0$. Then the law of partial measurement tells us that the superposition describing the *entire system* must collapse to contain only those states that are consistent with Alice's measurement. There is only one such state in $|\tau\rangle$, $|10\rangle Z |\sigma\rangle$, and therefore, with certainty, Bob's qubit b is now in the state $Z |\sigma\rangle$. Bob knows that Z is its own inverse, or $ZZ = I$, so he can apply Z to this state to get $ZZ |\sigma\rangle = |\sigma\rangle$.

Voilà, Bob's qubit b has the state $|\sigma\rangle$. Teleportation achieved!

As promised, there's been no cloning. In order for Alice's signal $|\sigma\rangle$ to make it to Bob's qubit, Alice had to measure both s and a . The process of measuring s collapsed it to either $|0\rangle$ or $|1\rangle$, destroying Alice's copy of $|\sigma\rangle$, thereby enabling us to move $|\sigma\rangle$ to Bob's qubit without cloning.

The Teleportation Process

We can think of teleportation as a four-step process. This will enable us to view the big picture in four smaller chunks that we can then assemble at the end.

The four steps are how Alice and Bob make the teleportation state $|\tau\rangle$, how Alice measures the state of the qubits, how Alice tells Bob which of the four states $|\tau\rangle$ collapsed to, and how Bob applies the correct qugates to get the original $|\sigma\rangle$. Let's take these in order.

Building $|\tau\rangle$

Alice and Bob build the teleportation state $|\tau\rangle$ in three steps. The first step entangles the qubits a and b , the second entangles s with the other two qubits, and the third performs one final step of processing.

Alice and Bob start everything off while they're still together on Earth. After lunch one day, they head to Alice's lab to make two qubits, named a and b , both in state $|0\rangle$. They entangle a and b in the same way that we saw in Figure 5-26. The traditional way to write this in a teleportation circuit is for Bob to apply H to b and then apply a CX using b as a control and a as a target, as shown in Figure 7-3(a).

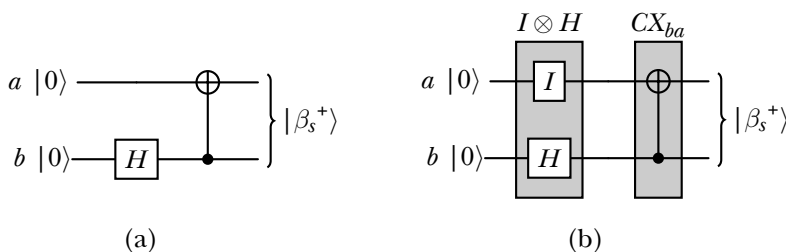


Figure 7-3: (a) Entangling a and b . (b) Explicitly including an identity on a and naming the operator systems.

This CX is drawn upside down compared to how I've usually drawn it before, with the control under the target. We saw this previously in Equation 5.48, where I called it CX' . We confirmed there that this works just as we'd hoped, with the target on the lower line controlling the application of the X qugate on the upper line. In this algorithm, I'll call the qugate CX_{ba} to emphasize that qubit b is controlling qubit a . As a reminder, the matrix form of CX_{ba} from Equation 5.48 is given in Equation 7.3.

$$CX_{ba} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \quad (7.3)$$

Returning to our entanglement step, our experience from Chapter 5 tells us that the output of Figure 7-3 should be $|\beta_s^+\rangle$. We can confirm this by first tensoring together $a = |0\rangle$ and $b = |0\rangle$ to make the starting state $|00\rangle$, then modifying them by the system $I \otimes H$ shown in Figure 7-3(b). This qugate system is written out in Equation 7.4.

$$I \otimes H = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \otimes \vee \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \vee \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{bmatrix} \quad (7.4)$$

The second system is CX_{ba} , which we just found. Let's apply both systems to the starting state $|00\rangle$, as shown in Equation 7.5.

$$\begin{aligned} CX_{ba}(I \otimes H) |00\rangle &= CX_{ba}((I \otimes H) |00\rangle) && \text{Apply } I \otimes H \text{ first} \\ &= CX_{ba} \left(\vee \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \right) && \text{Use } I \otimes H \text{ from Eq. 7.4} \\ &= CX_{ba} \vee \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} && \text{Multiply the matrices} \\ &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \vee \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} && \text{Use } CX_{ba} \text{ from Eq. 7.3} \\ &= \vee \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} && \text{Multiply the matrices} \\ &= \vee(|00\rangle + |11\rangle) = |\beta_s^+\rangle && \text{The Bell state } |\beta_s^+\rangle \end{aligned} \quad (7.5)$$

Great! Figure 7-3 does indeed give us the Bell state $|\beta_s^+\rangle$.

In our thought experiment, Bob now places qubit b in a special bottle and takes it with him to Mars. Alice also places a in a special bottle and puts it somewhere safe in her lab.

A year passes. One day, Alice completes her experiment, resulting in a qubit named s in the state $|\sigma\rangle$. This is the state she wants to send to Bob.

Alice can only modify s and a , the qubits that she has with her on Earth. In order for Alice to cause operations on the qubits s or a to affect Bob's qubit b far away, qubits s and a need to be entangled with b . She can create that entanglement by using s as the control on a CX targeting either a or b . Of these, only a is in the lab with Alice, so she entangles s with a . This second entanglement step is shown in Figure 7-4. I've written CX_{sa} for the CX in the usual orientation, using s as a control on a .

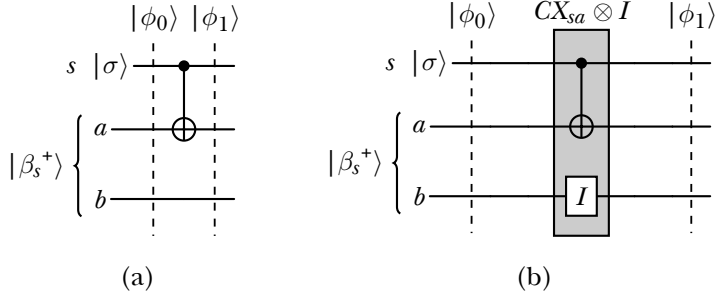


Figure 7-4: (a) Entangling s and a . (b) Including the implied I qugate.

Because we're focusing here on matrix elements, let's write out the components of the qubit system $|\phi_0\rangle$, the system just before Alice entangles s with a . This is $|\sigma\rangle$, the state of Alice's qubit s , tensored with $|\beta_s^+\rangle$, which we derived in Equation 7.5. The result is shown in Equation 7.6.

$$|\phi_0\rangle = |\sigma\rangle \otimes |\beta_s^+\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \otimes \vee \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \vee \begin{bmatrix} \alpha \\ 0 \\ \beta \\ 0 \\ 0 \\ \beta \end{bmatrix} \quad (7.6)$$

Now we'll entangle s with a . Figure 7-4(a) shows using s as a control on a . As usual, it omits the identity qugate I we could place on the b line. But although it's not in the picture, that identity must be in our operator!

Let's write out this matrix, as shown in Equation 7.7. Because CX_{sa} applies a control on the topmost line to a target immediately below it, its matrix is the familiar CX matrix.

$$CX_{sa} \otimes I = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \quad (7.7)$$

Now we can apply the system $CX_{sa} \otimes I$ to the state $|\phi_0\rangle$ we found in Equation 7.6 to get $|\phi_1\rangle$, as shown in Equation 7.8.

$$|\phi_1\rangle = (CX_{sa} \otimes I) |\phi_0\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \alpha \\ 0 \\ 0 \\ \alpha \\ \beta \\ 0 \\ 0 \\ \beta \end{bmatrix} = \begin{bmatrix} \alpha \\ 0 \\ 0 \\ \alpha \\ 0 \\ \beta \\ \beta \\ 0 \end{bmatrix} \quad (7.8)$$

I'll be pragmatic now. Looking at $|\phi_1\rangle$ in Equation 7.8, what would it take to turn this into the teleportation state $|\tau\rangle$ in Equation 7.1?

The trick is to write out $|\tau\rangle$ as a single state and compare it to $|\phi_1\rangle$ in Equation 7.8. Then we'll see if we can find a sequence of operations that juggle around the elements of $|\phi_1\rangle$ so that they match $|\tau\rangle$.

I'll find this explicit form of $|\tau\rangle$ in two steps. First, I'll expand each basis state $|00\rangle$ through $|11\rangle$ into its corresponding four-element ket, and then I'll replace each modified version of $|\sigma\rangle$ with the coefficients of its matrix.

As Equation 7.1 shows, we'll need four transformations in all, one for each basis state.

Equation 7.9 summarizes those four transformations for reference. In the fourth row, I applied Z and X operators (in that order) to make the combined operator XZ.

$$\begin{aligned}
 I|\sigma\rangle &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \\
 X|\sigma\rangle &= \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \beta \\ \alpha \end{bmatrix} \\
 Z|\sigma\rangle &= \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \alpha \\ -\beta \end{bmatrix} \\
 XZ|\sigma\rangle &= \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} -\beta \\ \alpha \end{bmatrix}
 \end{aligned} \tag{7.9}$$

With these in hand, let's rewrite $|\tau\rangle$. The steps are in Equation 7.10. The first line repeats $|\tau\rangle$ from Equation 7.1. The second line expands the basis states into kets, and the third replaces each modified version of $|\sigma\rangle$ with its state from Equation 7.9. In this last line, the rules of operator precedence tell us to perform the tensor operations before the additions.

$$\begin{aligned}
 |\tau\rangle &= \frac{1}{2} (|00\rangle I|\sigma\rangle + |01\rangle X|\sigma\rangle + |10\rangle Z|\sigma\rangle + |11\rangle XZ|\sigma\rangle) \\
 &= \frac{1}{2} \left(\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} I|\sigma\rangle + \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} X|\sigma\rangle + \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} Z|\sigma\rangle + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} XZ|\sigma\rangle \right) \\
 &= \frac{1}{2} \left(\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} \alpha \\ \beta \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} \beta \\ \alpha \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} \alpha \\ -\beta \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} -\beta \\ \alpha \end{bmatrix} \right)
 \end{aligned} \tag{7.10}$$

Finally, let's explicitly compute the tensor products in the last line of Equation 7.10, giving us Equation 7.11.

$$|\tau\rangle = \frac{1}{2} \left(\begin{bmatrix} \alpha \\ \beta \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \beta \\ \alpha \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ \alpha \\ -\beta \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ -\beta \\ \alpha \\ 0 \end{bmatrix} \right) = \frac{1}{2} \begin{bmatrix} \alpha \\ \beta \\ \beta \\ \alpha \\ \alpha \\ -\beta \\ -\beta \\ \alpha \end{bmatrix} \tag{7.11}$$

Great! Now we have an eight-element ket for $|\tau\rangle$. This is the very same τ we originally saw in Equation 7.1, but it's now represented as a single state vector, where all three qubits are entangled together.

This $|\tau\rangle$ is our goal. We want to turn the $|\phi_1\rangle$ in Equation 7.8 into this $|\tau\rangle$. How can we do this? Is there some operator A (or some sequence of operators that we can multiply together to make A) that we can plug into Equation 7.12 to do the trick?

$$A |\phi_1\rangle = |\tau\rangle, \quad \text{or} \quad \forall A \begin{bmatrix} \alpha \\ 0 \\ 0 \\ \alpha \\ 0 \\ \beta \\ \beta \\ 0 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} \alpha \\ \beta \\ \beta \\ \alpha \\ \alpha \\ -\beta \\ -\beta \\ \alpha \end{bmatrix} \quad (7.12)$$

Happily, we can indeed build an operator A that does just what we want. If you write down Equation 7.12 with a big empty matrix for A , then you can work through each element and fill in the entries. You'll find some elements must be 1, others -1 , and still others must be 0. And to turn the $\forall$ in $|\phi_1\rangle$ into the $1/2$ in $|\tau\rangle$, the matrix will need to include another factor of $\forall$. The resulting matrix is shown in Equation 7.13.

$$\forall \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & -1 \end{bmatrix} = \forall \left[\begin{array}{c|c} I_4 & I_4 \\ \hline I_4 & -I_4 \end{array} \right] \quad (7.13)$$

You can pull this pattern apart, as shown by the block matrix on the right. In the upper-left, upper-right, and lower-left blocks, we have the 4-by-4 identity matrix given by $I_4 = I \otimes I$. In the bottom-right block, we have its negative, $-(I \otimes I)$. This pattern of positives and negatives is just what we get from forming $H \otimes (I \otimes I)$. The parentheses aren't needed, but I've put them there to emphasize that we're thinking of $I \otimes I$ as one matrix that gets tensored with H .

In other words, the matrix in Equation 7.13 that takes us from $|\phi_1\rangle$ to $|\tau\rangle$ is given by $H \otimes I_4$, or $H \otimes I \otimes I$. In the circuit diagram, we draw an H on the top line and usually just imply the I qugates, as in Figure 7-5(a). Figure 7-5(b) shows what it looks like with the I qugates drawn explicitly.

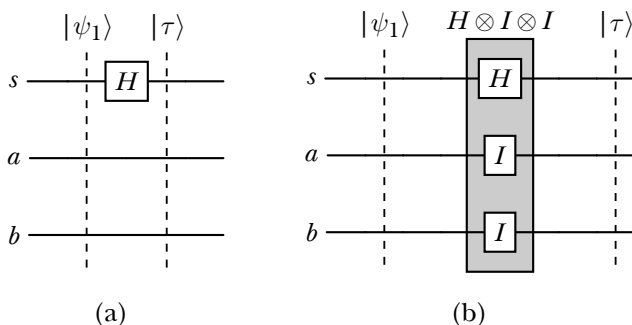


Figure 7-5: (a) Applying the final H to the top qubit, giving us the teleportation state $|\tau\rangle$. (b) Including the implied I qugates.

This process demonstrates that sometimes we build up a circuit from principles, and sometimes we just create something that performs a specific task.

Let's put it all together. Starting with qubits a and b in the state $|0\rangle$, we entangle them with an H and CX_{ba} , then we apply CX_{sa} , and finally we apply an H to s to create the teleportation state $|\tau\rangle$. The process is shown in Figure 7-6.

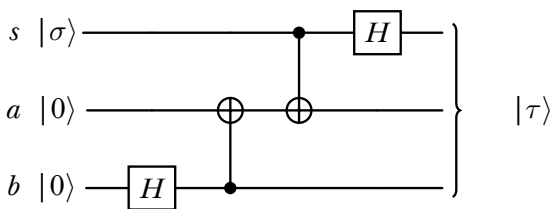


Figure 7-6: The full setup step for teleportation

This is the conceptual peak of the argument: We've created the teleportation state $|\tau\rangle$. The hard work is done! Now we're on the downhill slope from Figure 7-1.

Alice Measures Her Qubits

Now that the teleportation state has been set up, Alice will collapse it to just one state. This will push the state $|\sigma\rangle$ onto Bob's qubit (because it will have nowhere else to go) and simultaneously collapse the qubit s that has held $|\sigma\rangle$ until now.

Alice can measure her two qubits s and a in either order, or even at the same time. Appending this measurement to our existing circuit from Figure 7-6 gives us Figure 7-7. I'm labeling the output bits with the letter m (for measurement) rather than my usual b (for bit) because we're already using b for Bob's qubit.

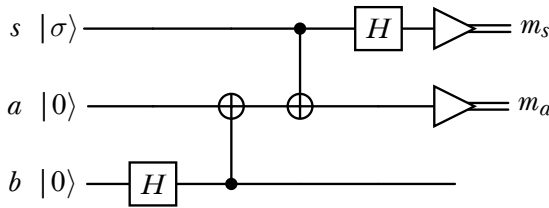


Figure 7-7: After the circuit of Figure 7-6, Alice measures her qubits.

Let's write the measured values from this system as a bitstring $m_s m_a$.

Suppose that Alice measures $m_s = 0$ and $m_a = 1$, or the bitstring 01. The law of partial measurement says that the system state $|\tau\rangle$ must collapse to states that are consistent with this measurement. That is, the system collapses to include only those states that start with $|01\rangle$. There is only one such state in $|\tau\rangle$ from Equation 7.1, and that's $|01\rangle X |\sigma\rangle$. So in this case, after the measurement, Alice's qubits a and s are now $|0\rangle$ and $|1\rangle$ respectively, and Bob's qubit b must be $X |\sigma\rangle$.

Alice Tells Bob the Measurements

When Alice's measurements are complete, our three-qubit system has collapsed to one of the four states in Equation 7.1. That is, depending on what Alice measured, Bob is holding a qubit that is in the state $I |\sigma\rangle$, or in the state $X |\sigma\rangle$, or in the state $Z |\sigma\rangle$, or in the state $XZ |\sigma\rangle$.

If Bob can determine the state of his qubit, he can apply the correct qugates to leave him with $|\sigma\rangle$. So the big question is, how can Bob tell which state he has?

That depends on what Alice measured. So the new question is, how can Bob determine what Alice saw on her meters?

The answer is that he can't! *Alice has to tell him.*

There's no quantum way for Alice to tell Bob anything, because Alice has already collapsed their entangled states.

The only remaining way for Alice to tell Bob what she measured is by using classical means. She can send the values of her two measurements via radio, or she can use a laser to bounce light off the moon, or she can send him a newspaper with the measurements printed somewhere. No matter how she chooses to send this information to Bob, she needs to use classical means, which are limited by the speed of light.

Bob Recovers $|\sigma\rangle$

Once Bob receives the two classical bits that Alice sent, telling him what she measured, he can recover $|\sigma\rangle$ from his qubit. His work is made easier by another remarkable feature of the structure of the teleportation state: Bob will need only two qugates!

The idea is that Bob will use the received classical bits as controls on a controlled-X qugate and a controlled-Z qugate. We've only discussed using quantum bits as controls, but we can use classical bits as well. Think of initializing a pair of qubits using the corresponding bits.

To see what Alice's measurements tell Bob, I've repeated the teleportation state $|\tau\rangle$ from Equation 7.1 here as Equation 7.14.

$$|\tau\rangle = \frac{1}{2} \left(|00\rangle I |\sigma\rangle + |01\rangle X |\sigma\rangle + |10\rangle Z |\sigma\rangle + |11\rangle XZ |\sigma\rangle \right) \quad (7.14)$$

Bob's actions to recover $|\sigma\rangle$ are shown in Figure 7-8.

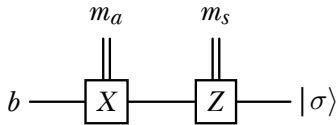


Figure 7-8: Bob decoding his qubit based on Alice's classical bits

Let's look at the four possible pairs of bits that Bob might receive.

If Alice measured 00, then $m_a = m_s = 0$. Equation 7.14 tells us that when Alice measures 00, Bob has $I |\sigma\rangle = |\sigma\rangle$. Since both controls are 0, neither qugate is applied. This is just right, because Bob already has $I |\sigma\rangle = |\sigma\rangle$, and he's done.

If Alice measured 01, then $m_s = 0$ and $m_a = 1$. Equation 7.14 tells us that Bob's qubit is in state $X |\sigma\rangle$. Bob applies the controlled-X qugate, and because X is its own inverse, this gives him $XX |\sigma\rangle = |\sigma\rangle$, and he's recovered $|\sigma\rangle$.

If Alice measured 10, then $m_s = 1$ and $m_a = 0$. This tells Bob to apply the controlled-Z qugate. Like the X qugate, Z is its own inverse, so Bob gets $ZZ |\sigma\rangle = |\sigma\rangle$.

Finally, If Alice measured 11, then $m_a = m_s = 1$, and Bob knows that his qubit is $XZ |\sigma\rangle$. To undo the transformation XZ , Bob applies X and then Z in that order, which is equivalent to applying the single operator ZX (remember to read the operators from right to left). The steps are shown in Equation 7.15.

$$\begin{aligned} ZX(XZ) |\sigma\rangle &= Z(XX)Z |\sigma\rangle && \text{Regroup matrix multiplies} \\ &= ZZ |\sigma\rangle && \text{Since } XX = I \\ &= |\sigma\rangle && \text{Since } ZZ = I \end{aligned} \quad (7.15)$$

It's important to keep the order of the operations at each point in the algorithm clear in your mind. If Alice's qubit is in the state $XZ |\sigma\rangle$, then to recover $|\sigma\rangle$ Bob has to apply the inverse of operator XZ , which is ZX .

And we're done. For each of Alice's four possible measurements, Bob has successfully ended up with his qubit in the state $|\sigma\rangle$. We've teleported $|\sigma\rangle$ from Alice to Bob!

Drawing the Teleportation Protocol

Let's now put all the pieces together, giving us Figure 7-9.

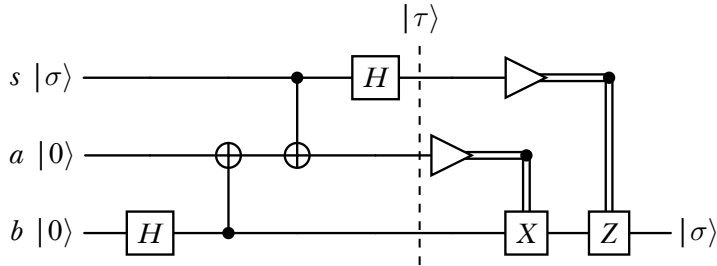


Figure 7-9: The full teleportation circuit as it's normally drawn, with the teleportation state $|\tau\rangle$ marked

Figure 7-9 is the complete, traditional quantum teleportation protocol. This theoretical process has been experimentally confirmed [22].

However, Figure 7-9 may be somewhat misleading, because it shows all three qubits at the far left, suggesting that Alice and Bob have them all in their control at the start of the process. But if Alice already has s in the state $|\sigma\rangle$, and Bob is standing there, she could just hand the s qubit (protected carefully) to Bob, and there would be no need for teleporting anything!

For that reason, I prefer drawing this as in the overall recap of Figure 7-10. The delayed introduction of s clarifies that s isn't yet in the state $|\sigma\rangle$ when Alice and Bob are entangling a and b . Only later does Alice compute s , and then continue the protocol.

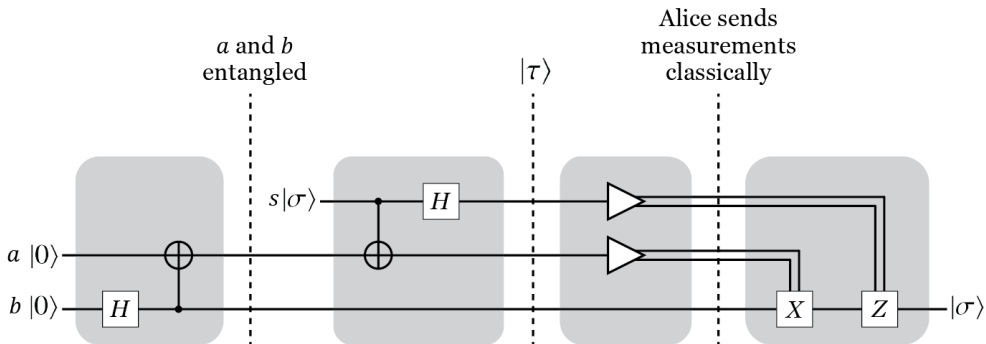


Figure 7-10: A recap of the quantum teleportation algorithm

Teleportation is usually drawn as in Figure 7-9, so keep in mind that in practice, the qubit s is usually not in the state $|\sigma\rangle$ at the start, when Alice and Bob are creating their entangled pair.

Probabilistic Teleportation

We've seen that after Alice has measured her qubits, Bob's qubit is in one of the four states in the superposition $|\tau\rangle$, but he doesn't know which one. Alice has to tell him by sending him two classical bits by classical means.

But let's suppose that for some reason, Alice can't send Bob her bits. Is the situation hopeless for Bob, or is there some way, perhaps with a combination of effort and luck, that he will be able to recover $|\sigma\rangle$ from his qubit?

Let's try another thought experiment.

Suppose that there's been a terrible accident on Mars. The habitat blew up when Bob was out on a mission, leaving Bob the only survivor. The explosion also damaged the rocket he and his colleagues were going to use to return to Earth, and almost all the fuel has leaked out. There's enough fuel to lift off the surface, but there isn't nearly enough to get the rocket back to Earth.

Bob's supplies will run out long before a rescue mission can reach him, so he needs to find a new way home.

Luckily, an earlier Mars mission placed an emergency rescue satellite in Mars orbit. If Bob can reach it, the resources there will not only keep him alive, but he'll also be able to repair his rocket. And there's enough fuel there to fill the rocket's tanks. It's a plan to get home!

Getting his damaged rocket safely up to the rescue satellite will require an elaborate flight plan with multiple steps that he'll have to perform at the right moments. The specific plan for any given day will depend a lot on the local weather.

Unfortunately, Bob doesn't have access to the weather satellites above Mars. But he does have a working radio, and he contacts Alice, who can read the weather satellite data without a problem. They agree that Alice will use that data to work out a flight plan for the next day and send it to Bob. Because these flight plans are complicated, she'll encode the entire plan into a single state, $|\sigma\rangle$. This is a good strategy for them, because before Bob left, he and Alice created lots and lots of entangled pairs to use for teleporting states over the duration of Bob's mission. Bob's half of each pair survived the accident. So once Alice tells him her measurements, Bob can take down the bottle containing the next qubit to be used and apply Alice's bits to put it into the state $|\sigma\rangle$.

On Mars, Bob has cobbled together a decoder to turn a quantum state sent by Alice into a flight plan. He's also written a simulation program that will look at a flight plan and tell him whether it's safe and he'll reach the satellite, or it's unsafe and the rocket will blow up, along with Bob.

The next morning, as Bob prepares to hear from Alice what her measurements were, Bob's radio won't even turn on. It's busted, and he doesn't have the parts to fix it, so now he's lost all touch with Alice. He can't get her measurements, so he can't confidently process his qubit to turn it into $|\sigma\rangle$.

Bob isn't completely without hope, though. He can just plain old guess. Suppose he guesses that Alice measured 00, so his qubit is in the state $I|\sigma\rangle$, and he doesn't have to process it. He has a three in four chance of being wrong, but more optimistically, a one in four chance of being right!

So he feeds his qubit into the decoder, which gives him a flight plan. He then gives that to the safety testing program. If he's lucky, the test will tell him that he guessed correctly and that the flight plan makes sense and is safe. But if he's unlucky, then his guess was wrong (that is, his qubit wasn't in the state $|\sigma\rangle$, but one of the other states in $|\tau\rangle$). In that case, the decoder will have produced a nonsensical flight plan, and the test will tell him that following that plan would end in disaster.

If Alice and Bob were sharing only a single entangled pair, this would be the end of the story. Bob would have no option but to launch the rocket anyway and hope for the best.

Is there some way, *any way*, that Bob can improve his odds?

Recall that Alice and Bob created not one entangled pair before Bob left, but many dozens or hundreds of them. They intended to use them to teleport different quantum states over the course of Bob's mission. But now that Alice knows Bob's radio is out, she'll use them all right away. Alice will run the quantum protocol to teleport the same state $|\sigma\rangle$ over every pair of entangled qubits. Since she can't clone the $|\sigma\rangle$ she's made, she runs her plan-making program many times, creating many distinct qubits that are all $|\sigma\rangle$. She plugs each of these qubits into the teleportation protocol, and even measures the output bits, though she can't send them to Bob. At this point, she's done all she can.

Back on Mars, suppose that Bob's guess for his first qubit resulted in a meaningless and unsafe flight plan. He hopes that Alice is following their backup plan, and gives her some time to compute and entangle $|\sigma\rangle$ on all of their remaining pairs.

After a little while, he'll take down the next of his entangled qubits, guess again, and process the qubit according to his guess of what state the qubit is in (he could just guess it's $|\sigma\rangle$ every time, and apply no qugates to it). Then Bob will decode his qubit and test the resulting flight plan, hoping it will be safe.

The process is shown graphically in Figure 7-11. Alice measures s and a , producing classical bits m_s and m_a , but she does nothing with them. The measurements were just to collapse the states of the qubits. Bob then feeds his qubit b into the decoding algorithm G that turns that qubit into a flight path, represented by the binary number g that comes from measuring the output of G . Because g is a classical binary number, Bob can make as many copies of it as he pleases.

So, Bob makes a copy of g and feeds it into his test, along with whatever other inputs it needs. If the test says the flight plan is safe, then he can follow the steps in g and he's all set for launch!

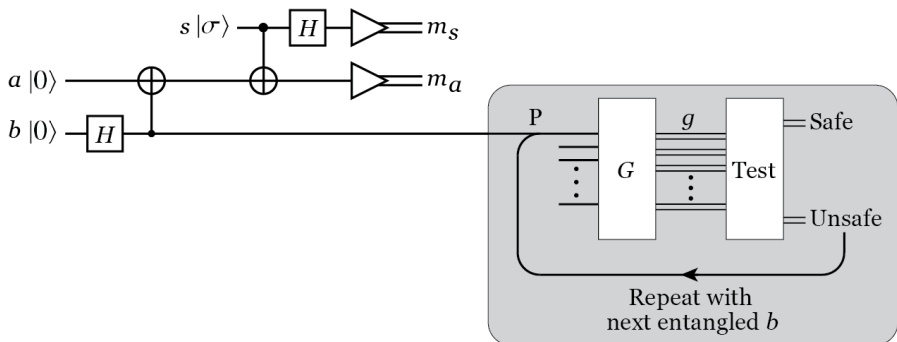


Figure 7-11: After Alice has measured her qubits, Bob guesses that he has $|\sigma\rangle$, computes the binary bitstring g , and then tests that bitstring to see if it's a safe flight plan.

If the test tells him that the flight plan is unsafe, he takes down his next qubit b , makes another guess, and tries again.

Because there's no way for Bob to be sure beforehand that a given qubit actually has the value he's guessing, we might call this process *probabilistic teleportation*.

An important thing to keep in mind is that Bob doesn't really care about the state $|\sigma\rangle$. He'll *use* each qubit, rather than study it, and it's the results of the decoder and test that he cares about.

Once Bob has guessed at the state of any qubit b , he has a one in four chance, or a probability of 0.25, that he'll have guessed correctly and the test will tell him the plan is safe. Those aren't great odds.

What are the chances that when Bob uses this approach, he will ultimately guess right, and thereby get a safe flight plan that could save his life?

To see Bob's chances of success, consider his odds of failure. After one guess (and any processing it might require), there's a three in four chance that b is *not* in the state $|\sigma\rangle$. Thus, Bob has a 0.75 probability of being wrong (and getting an unsafe plan). But this means he has a $1 - 0.75 = 0.25$ probability of being right (and getting a safe plan). After two repeats of the teleportation, his probability of guessing incorrectly both times is $0.75 \times 0.75 = 0.75^2 = 0.5625$. Thus, his probability of being correct at least once is $1 - 0.5625 = 0.4375$. Much better!

After n repeats of the protocol, the probability that Bob has guessed correctly at least once is $1 - 0.75^n$, which I've graphed in Figure 7-12.

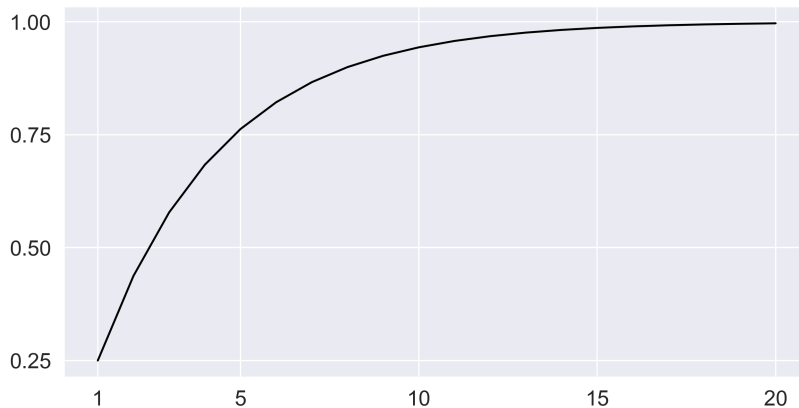


Figure 7-12: A plot of $1 - 0.75^n$ for n from 1 to 20

After 10 repeats, Bob has about a 0.94 probability of having been right at least once. After 20 repeats, his probability of having been right at least once is almost 0.997.

If Alice and Bob are willing to share 20 entangled qubits before Bob leaves, *and* Alice computes her side of the protocol from scratch 20 times, *and* Bob runs his decoder and test 20 times, there's about a 99.7 percent likelihood that Bob will have guessed correctly at some point and obtained a safe flight plan. He only needs one safe flight plan, and he can stop as soon as he has it.

After 50 attempts, Bob's chance of never guessing right, even once, is less than 1 in a million. But even though the odds of Bob guessing correctly go up with each repeat, if he's super unlucky, he might never guess correctly. Worse, Bob has no way to tell Alice if he's guessed correctly or not.

When Alice and Bob are able or willing to share two classical bits, they can run the classical protocol once and teleport the qubit for sure. If they don't want to share those classical bits, or they're unable to, they can hope to teleport the qubit, but they'll have to put in a lot of extra effort, and they might still fail.

To be guaranteed success, Alice needs to send the two classical bits representing her measurements to Bob, using classical means. If they can't do that, they can hope that luck is on Bob's side and he'll get an answer that passes his test before he runs out of qubits.

This is only one of many interesting modifications to the basic teleportation algorithm [4] [158] [289]. You can read up on the references, or try your own ideas. Exploring variations on circuits you already understand is a great way to gain experience with quantum algorithms.

Summary

The teleportation protocol lets us transfer a quantum state from one qubit to another, which can be arbitrarily far away. The process requires that Alice and Bob already share an entangled pair of qubits, and that Alice can transmit two classical bits to Bob. Alice's measurements cause qubit s to collapse. This means that there is never more than one qubit in the state $|\sigma\rangle$.

The big surprise of teleportation is that the state we've transferred contains two complex numbers, each of which is built from two real numbers, for a total of four real numbers. These numbers can require arbitrary numbers of digits if written out, but they will be transferred with perfect precision.

Once Bob has operated on his state, he can measure it. As always, measurement will give him only a 0 or a 1, so there's no way to extract those four real numbers that were transmitted. But before measurement, Bob can use his qubit, now in state $|\sigma\rangle$, in further computations. So Bob can then build on Alice's work, using her result as an input to his own algorithm.

Although the collapse of Bob's qubit is immediate after Alice's measurements, the need to then share classical bits prevents us from using this protocol to send information faster than the speed of light. That's too bad, but it doesn't change the fact that quantum teleportation is still a pretty amazing feat.

If Alice and Bob have lost all classical communication, and they have some additional resources, they can use a probabilistic approach that is likely to give Bob the state $|\sigma\rangle$ eventually. But he'll probably have to try several different instances of b (independently computed by Alice) and then test each result.

In this chapter, we looked at the matrix elements behind Alice's computation of the teleportation state $|\tau\rangle$, and then we unpacked that state to teleport $|\sigma\rangle$ to Bob. We had to deal with a few big matrices, but they

were manageable. In general, a system of n qubits will need an operator described by a matrix 2^n elements on a side, which quickly becomes too big to write out and manually compute with. So from now on, we'll focus on the algebraic approach most of the time, rather than writing out the matrices and kets.

Teleportation is an amazing algorithm, and it shows the power of entanglement for sharing information at a great distance. It's pretty incredible that Alice can perform some quantum operations on qubits on Earth and change the states of Bob's qubits on Mars (or even a planet in orbit around a star millions of light years away).

In the next few chapters, we'll look at more quantum algorithms, each of which introduces one or more new concepts into our quantum repertoire.

APPENDIX: NOTATION

Nothing is more easy than the invention of notation, and nothing of worse example and consequence than the confusion of mathematical expressions by unknown symbols.

—Augustus De Morgan, *A Treatise on the Calculus of Functions*, 1836 [45]

Oh no. No. No. I meant no scheme. I merely posed a little academic accounting theory. It was just a thought.

—Mel Brooks, *The Producers*, 1967 [28]

Table A-1: Constants

e	Euler’s number, 2.7182...
i	Imaginary number, defined as $\sqrt{-1}$
π	The ratio of a circle’s circumference to its radius, 3.1415...
$\vee$	$1/\sqrt{2} = \sqrt{1/2} = \sqrt{2}/2$

Table A-2: Sets

$\mathbb{B}$	The set of bits (0 and 1)
$\mathbb{C}$	The set of complex numbers
$\mathbb{N}$	The set of natural (or counting) numbers
$\mathbb{R}$	The set of real numbers
$\mathbb{Z}$	The set of integer numbers
$\mathbb{R} \times \mathbb{R}$	The set of all pairs of real numbers
$\mathbb{R}^n$	The set of all tuples of n real numbers
$\{a, b, c \dots\}$	A set of elements
$a \in \mathbb{X}$	a belongs to set $\mathbb{X}$
$\mathbb{B}^n$	A bitstring of n elements

Table A-3: Sequences and ranges

$[a, b]$	The integers from a up to and including b
(a, b)	The integers from $a + 1$ up to and including $b - 1$
$[d]$	The sequence of integers from 0 up to and including $d - 1$

Table A-4: Common symbols

$\oplus$	Exclusive OR
$\otimes$	Tensor product
$A^{\otimes n}$	An operator A tensored with itself n times
$ \psi\rangle^{\otimes n}$	A state $ \psi\rangle$ tensored with itself n times
0^n	A bitstring of n bits, all 0s
1^n	A bitstring of n bits, all 1s
$a \cdot b$	For bitstrings a and b , $a_0b_0 \oplus a_1b_1 \oplus \dots \oplus a_{n-1}b_{n-1}$
$\delta_{j,k}$	Kronecker delta for integers j, k ; returns 1 if $j = k$, else 0

Table A-5: Common variables

d	The dimensionality (or number of elements) of a vector
n	Often the number of qubits in a circuit
N	2^n for a given n
k	A generic integer
θ	When discussing geometry, a generic angle
$ \psi\rangle, \phi\rangle$	Generic kets
$\langle\psi , \langle\phi $	Generic bras
ω	$e^{i2\pi/N}$
λ	$e^{i2\pi}$

Table A-6: Common variable types by typography

b	Roman lowercase italic b : a bit or bitstring
r	Roman lowercase italic: a real or integer
A	Roman uppercase italic: a list, set, operator, or matrix
α	Greek lowercase: a complex number
$\mathbf{v}$	Roman lowercase bold: a vector

Table A-7: Numbering and counting

1	First number when counting
0	First number when numbering
Top down	Order for numbering elements in a column vector
Left to right	Order for numbering elements in a row vector
Top down	Order for tensoring operators in a circuit diagram
Top down	Order for tensoring states in a circuit diagram

Table A-8: XOR properties

$x \oplus x = 0$	X1
$x \oplus 0 = x$	X2
$(x \oplus y) \oplus z = x \oplus (y \oplus z)$	X3

Table A-9: Complex numbers

$\bar{\alpha}$	The complex conjugate of α
α^*	An alternative form of $\bar{\alpha}$
$a + bi$	The Cartesian form of a complex number
$re^{i\theta}$	The polar form of a complex number

Table A-10: Vectors, quantum states, and operations

$\mathbf{a} \cdot \mathbf{b}$	The dot product of $\mathbf{a}$ and $\mathbf{b}$
$\langle \mathbf{a}, \mathbf{b} \rangle$	The inner product of $\mathbf{a}$ and $\mathbf{b}$
$ \psi\rangle$	A ket (a complex column matrix)
$\langle\psi $	A bra (a complex row matrix)
$\langle\phi \psi\rangle$	A bracket (or inner product) of $ \phi\rangle$ and $ \psi\rangle$
$ \phi\rangle\langle\psi $	A ketbra (or outer product) of $ \phi\rangle$ and $ \psi\rangle$
$\langle\text{onto}, \text{from}\rangle$	Projection using the inner product

Table A-11: Properties of vectors

$ \mathbf{v} $	Magnitude of a vector
$\dim(\mathbf{v})$	Dimensionality of a vector

Table A-12: Vector and linearity properties

$\sigma\mathbf{v} = \sigma \begin{bmatrix} \nu_0 \\ \nu_1 \\ \nu_2 \end{bmatrix} = \begin{bmatrix} \sigma \nu_0 \\ \sigma \nu_1 \\ \sigma \nu_2 \end{bmatrix}$	V1
$\mathbf{v} + \mathbf{w} = \begin{bmatrix} \nu_0 \\ \nu_1 \\ \nu_2 \end{bmatrix} + \begin{bmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \end{bmatrix} = \begin{bmatrix} \nu_0 + \omega_0 \\ \nu_1 + \omega_1 \\ \nu_2 + \omega_2 \end{bmatrix}$	V2
$f(\sigma\mathbf{v}) = \sigma f(\mathbf{v}), \quad \sigma \in \mathbb{C}$	L1
$f(\mathbf{v} + \mathbf{w}) = f(\mathbf{v}) + f(\mathbf{w})$	L2

Table A-13: Matrix operations

$\bar{A}$	The complex conjugate of matrix A
A^T	The transpose of matrix A
$A^\dagger$	The adjoint of matrix A
A^{-1}	The inverse of matrix A
AB	Ordinary matrix multiplication of matrices A and B

Table A-14: Tensor product properties

$\overline{A \otimes B} = \overline{A} \otimes \overline{B}$	Conjugate distributivity
$(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$	Tensor inverse property
$(A \otimes B)^T = A^T \otimes B^T$	Tensor transpose property
$(A \otimes B)^\dagger = A^\dagger \otimes B^\dagger$	Tensor adjoint property
$A \otimes (B + C) = (A \otimes B) + (A \otimes C)$	Tensor product is distributive
$A \otimes (B \otimes C) = (A \otimes B) \otimes C$	Scalar tensor property
$c(A \otimes B) = (cA) \otimes B$ $= A \otimes (cB)$	For any $c \in \mathbb{R}$
$(A \otimes B)(C \otimes D) = AC \otimes BD$	Mixed-product property

Table A-15: Bitstring operations

$a \cdot b = \left(\sum_{k \in [d]} a_n \oplus b_n \right) \bmod 2$	Bitstring dot product
$a \oplus b = a_{n-1} \oplus b_{n-1}, \dots, a_0 \oplus b_0$	Bitstring XOR

Table A-16: Common states

$ 0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad 1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad +\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad -\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

Table A-17: Integer bras and kets

$ k\rangle$	For integer k , a ket of all 0 elements except a 1 at index k
$\langle k $	A bra of all 0 elements except a 1 at index k , or $ k\rangle^\dagger$

Table A-18: Superpositions

$\alpha \phi\rangle + \beta \psi\rangle$	Superposition of 2 states, $ \alpha ^2 + \beta ^2 = 1$
$\sum_{k \in [n]} \alpha_k \phi_k\rangle$	Superposition of n states, $\sum_{k \in [n]} \alpha_k ^2 = 1$
$\bigvee^n \sum_{k \in [n]} k\rangle$ $= H^{\otimes n} 0\rangle^{\otimes n}$ $= +\rangle^{\otimes n}$	Uniform superposition of 2^n states

Table A-19: Operations on quantum states

$ \psi\rangle$	The magnitude of $ \psi\rangle$
$\langle\phi $	The magnitude of $\langle\phi $
$\langle\phi M \psi\rangle$	A bramket of $ \phi\rangle$, M , and $ \psi\rangle$
$\langle r A c\rangle$	Element $a_{r,c}$ for integers r and c
$\langle\phi A \psi\rangle = (\langle\phi A) \psi\rangle$ $= \langle\phi (A \psi\rangle)$	Matrix products are associative
$ \psi\phi\rangle = \phi\rangle \psi\rangle = \phi\rangle \otimes \psi\rangle$	Ways to write a product state

Table A-20: Hadamard operations on common states

$H 0\rangle = +\rangle$
$H 1\rangle = -\rangle$
$H +\rangle = 0\rangle$
$H -\rangle = 1\rangle$

Table A-21: Algebraic expressions for $H|j\rangle$

$H j\rangle = \vee \sum_{k \in \mathbb{B}} (-1)^{j \cdot k} k\rangle$	For $j \in \mathbb{B}$
$H j\rangle = \vee (0\rangle + e^{i2\pi(j/2)} 1\rangle)$	For $j \in \mathbb{B}$
$H j\rangle = \vee (0\rangle + \lambda^{j/2} 1\rangle)$	For $j \in \mathbb{B}$

Table A-22: Bell states

$ \beta_s^+\rangle = \Psi^+\rangle = \beta(0,0)\rangle = \vee (00\rangle + 11\rangle)$
$ \beta_m^+\rangle = \Phi^+\rangle = \beta(0,1)\rangle = \vee (01\rangle + 10\rangle)$
$ \beta_s^-\rangle = \Psi^-\rangle = \beta(1,0)\rangle = \vee (00\rangle - 11\rangle)$
$ \beta_m^-\rangle = \Phi^-\rangle = \beta(1,1)\rangle = \vee (01\rangle - 10\rangle)$

Table A-23: Measurement operators

Π_k	Projection matrix $ k\rangle\langle k $
Π_0	Projection matrix $ 0\rangle\langle 0 $
Π_1	Projection matrix $ 1\rangle\langle 1 $
$\Pr_{ \psi\rangle}(k)$	Probability that measuring $ \psi\rangle$ will return k

Table A-24: Eigenthings

ε_k	Eigenvector k
λ_k	Eigenvalue k

Table A-25: Common qugate matrices in the computational basis

I	The identity of any size; for example, $I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$
$CX, CNOT$	Controlled- X or controlled- NOT , $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$
CX'	CX with control below target, $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$
H	Hadamard, $\vee \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
P	Phase, $\begin{bmatrix} 0 & 1 \\ 1 & e^{i\theta} \end{bmatrix}$
S	$\pi/2$ phase, $\begin{bmatrix} 0 & 1 \\ 1 & e^{i\pi/2} \end{bmatrix}$
T	$\pi/4$ phase, $\begin{bmatrix} 0 & 1 \\ 1 & e^{i\pi/4} \end{bmatrix}$
$SWAP$	Swap, $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
U	Generic unitary, $\begin{bmatrix} \cos\left(\frac{\theta}{2}\right) & -e^{i\lambda} \sin\left(\frac{\theta}{2}\right) \\ e^{i\phi} \sin\left(\frac{\theta}{2}\right) & e^{i(\phi+\lambda)} \cos\left(\frac{\theta}{2}\right) \end{bmatrix}$

Table A-26: Pauli matrices

X , NOT	Pauli X , also σ_x , $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Y	Pauli Y , also σ_y , $\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Z	Pauli Z , also σ_z , $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

Table A-27: Common geometrical matrices

$\text{Rot}(\theta)$	Rotation by θ counterclockwise, $\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$
$\text{Ref}(\theta)$	Reflection by line θ counterclockwise from X axis, $\begin{bmatrix} \cos 2\theta & \sin 2\theta \\ \sin 2\theta & -\cos 2\theta \end{bmatrix}$

Table A-28: Circuit symbols: general

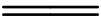
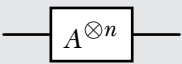
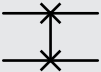
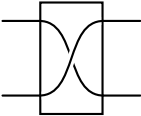
	A quantum wire
	A classical wire
	A group of n qubits
	A quantum gate implementing operator A
	A vertical system of n copies of qugate A
	Alternate form of an X qugate
	A two-qubit SWAP qugate
	Alternate form of a two-qubit SWAP qugate

Table A-29: Circuit symbols: measurement and entanglement

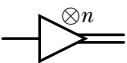
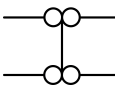
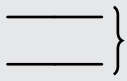
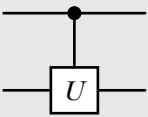
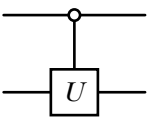
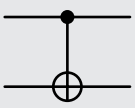
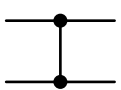
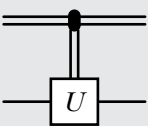
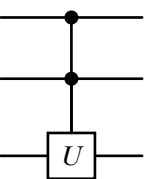
	A meter
	A system of n meters tensored together
	Alternate form of a meter
	An entangled pair
	Alternate form of an entangled pair

Table A-30: Circuit symbols: controlled qugates

	A controlled- U qugate enabled only if the control is $ 1\rangle$
	A controlled- U qugate enabled only if the control is $ 0\rangle$
	Common shortcut for a CX qugate
	Common shortcut for a CZ qugate
	A U qugate controlled by a classical bit
	A controlled- U qugate enabled only if both controls are $ 1\rangle$

BIBLIOGRAPHY

I dabbled in bird-watching once. Well, by dabbled, I mean I bought a bird guide and put it up on the shelf and looked up at it sometimes.

—Scott Jacobson, *Bob’s Burgers*, “Crows Encounters of the Bird Kind,” 2023 [100]

Whenever possible, I’ve tried to use free and accessible online references. I cited books and other traditional references when they are the superior choice or offer a unique perspective.

- [1] Ackerman, Jennifer, and Eleanor Grant. “Born to Move.” <https://www.imdb.com/title/tt1776626>, 2010.
- [2] Adams, Douglas. *Dirk Gently’s Holistic Detective Agency*. Pocket Books, 1987.
- [3] Adams, Douglas. *The Hitchhiker’s Guide to the Galaxy*. Pan Books, 1978.
- [4] Agrawal, Pankaj, and Arun K. Pati. “Probabilistic Quantum Teleportation.” <https://arxiv.org/pdf/quant-ph/0210004>, 2002.
- [5] Alves, Carolina, Luís Paulo Santos, and Thomas Bashford-Rogers. “A Quantum Algorithm for Ray Casting Using an Orthographic Camera.” In *2019 International Conference on Graphics and Interaction*, 2019. https://repositorium.sdum.uminho.pt/bitstream/1822/71756/3/Quantum_Algorithm.pdf.
- [6] Amazon Web Services. “Amazon Braket.” <https://aws.amazon.com/braket/>, 2025.
- [7] American Kennel Club. “Breed Weight Chart.” <https://www.akc.org/expert-advice/nutrition/breed-weight-chart/>, 2023.
- [8] Answers.com. “How Much Does a Ladybug Weigh?” https://www.answers.com/Q/How_much_does_a_ladybug_weigh, 2023.
- [9] Arute, Frank, Kunal Arya, Ryan Babbush, et al. “Quantum Supremacy Using a Programmable Superconducting Processor.” *Nature* 574 (2019): 505–10. <https://www.nature.com/articles/s41586-019-1666-5>.
- [10] Arvo, James, Robert L. Cook, Andrew Glassner, Eric Haines, Pat Hanrahan, Paul S. Heckbert, and David Kirk. *An Introduction to Ray Tracing*. Morgan Kaufmann, 1989.

- [11] Aumasson, Jean-Philippe. *Serious Cryptography: A Practical Introduction to Modern Encryption*. No Starch Press, 2017.
- [12] Azad, Kalid. “Understanding the Birthday Paradox.” *Better Explained* blog, 1997. <https://betterexplained.com/articles/understanding-the-birthday-paradox/>.
- [13] Babbush, Ryan, Dominic W. Berry, Robin Kothari, Rolando D. Somma, and Nathan Wiebe. “Exponential Quantum Speedup in Simulating Coupled Classical Oscillators.” *Physical Review X* 13 (2023): 041041.
- [14] Bacon, Dave. “CSE 599d—Quantum Computing: Simon’s Algorithm.” University of Washington, 2006. <https://courses.cs.washington.edu/courses/cse599d/06wi/lecturenotes8.pdf>.
- [15] Bartlett, Albert A. “The Exponential Function—Part 1.” *The Physics Teacher* 14, no. 7 (1976): 393–401. <https://doi.org/10.1119/1.2339436>.
- [16] Bennett, Charles H., Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. “Strengths and Weaknesses of Quantum Computing.” <https://arxiv.org/pdf/quant-ph/9701001>, 1997.
- [17] Berger, Cynthia. “True Colors: How Birds See the World.” <https://www.nwf.org/Magazines/National-Wildlife/2012/AugSept/Animals/Bird-Vision>, 2012.
- [18] Bernstein, Ethan, and Umesh Vazirani. “Quantum Complexity Theory.” *SIAM Journal on Computing* 26, no. 5 (1997): 1411–73. <http://wpape.unina.it/pieroandrea.bonatti/didattica/complexity/slides/Bernstein-Vazirani.pdf>.
- [19] Bertlmann, Reinhold, and Nicolai Friis. “Density Matrices.” In *Modern Quantum Theory: From Quantum Mechanics to Entanglement and Quantum Information*, 321–49. Oxford University Press, 2023.
- [20] Bethesda Game Studios. “Starfield.” <https://www.imdb.com/title/tt8545540>, 2023.
- [21] Beutlich, Robert E. “Radionics and the Little Black Box.” <https://altered-states.net/barry/update183/radionics2.htm>.
- [22] Bouwmeester, Dik, Jian-Wei Pan, Klaus Mattle, Manfred Eibl, Harald Weinfurter, and Anton Zeilinger. “Experimental Quantum Teleportation.” <https://arxiv.org/pdf/1901.11004>, 1997.
- [23] Boyer, Michel, Gilles Brassard, Peter Høyer, and Alain Tapp. “Tight Bounds on Quantum Searching.” *Fortschritte der Physik* 46, no. 4–5 (1998): 187–99. https://www.researchgate.net/publication/227992557_Tight_Bounds_on_Quantum_Searching.
- [24] Bradley, Tai-Danae. “The Tensor Product, Demystified.” <https://www.math3ma.com/blog/the-tensor-product-demystified>, 2018.
- [25] Brahic, Catherine. “Birds Can ‘See’ the Earth’s Magnetic Field.” *New Scientist*, 2008. <https://www.newscientist.com/article/dn13811-birds-can-see-the-earths-magnetic-field/>.

- [26] Brassard, Gilles, and Peter Høyer. “An Exact Quantum Polynomial-Time Algorithm for Simon’s Problem.” <https://arxiv.org/pdf/quant-ph/9704027.pdf>, 1997.
- [27] Brassard, Gilles, Peter Høyer, Michele Mosca, and Alain Tapp. “Quantum Amplitude Amplification and Estimation.” In *AMS Contemporary Mathematics Series*, vol. 305. American Mathematical Society, 2002. <https://arxiv.org/pdf/quant-ph/0005055>.
- [28] Brooks, Mel. *The Producers*, 1967. <https://www.imdb.com/title/tt0063462>.
- [29] Brooks, Michael. “How a Quantum Innovation May Quash the Idea of the Multiverse.” *New Scientist*, 2025. <https://www.newscientist.com/article/mg26435252-200-how-a-quantum-innovation-may-quash-the-idea-of-the-multiverse/>.
- [30] Cai, Guangya, and Daowen Qiu. “Optimal Separation in Exact Query Complexities for Simon’s Problem.” *Journal of Computer and System Sciences* 97 (2018): 83–93. <https://doi.org/10.1016/j.jcss.2018.05.001>.
- [31] Cartwright, Jon. “The Quantum World: A Concise Guide to the Particles That Make Reality.” <https://www.newscientist.com/article/2367423-the-quantum-world-a-concise-guide-to-the-particles-that-make-reality/>, 2023.
- [32] Carver, George Washington. “Quotefancy.” Approx. 1934. <https://quote fancy.com/george-washington-carver-quotes>.
- [33] Castelvechi, Davide. “Magnetic Sense Shows Many Animals the Way to Go.” *Scientific American*, 2012. <https://www.scientificamerican.com/article/the-compass-within/>.
- [34] Cherney, David, Tom Denton, and Andrew Waldron. “7.1: Linear Transformations and Matrices.” In *Linear Algebra*, LibreTexts. [https://math.libretexts.org/Bookshelves/Linear_Algebra/Map%3A_Linear_Algebra_\(Waldron_Chерney_and_Denton\)/07%3A_Matrices/7.01%3A_Linear_Transformations_and_Matrices](https://math.libretexts.org/Bookshelves/Linear_Algebra/Map%3A_Linear_Algebra_(Waldron_Chерney_and_Denton)/07%3A_Matrices/7.01%3A_Linear_Transformations_and_Matrices).
- [35] Chesser, Preston. “Oracle of Delphi.” <https://ehistory.osu.edu/articles/oracle-delphi>.
- [36] Childs, Andrew M., Matthew Coudron, and Amin Shiraz Gilani. “Quantum Algorithms and the Power of Forgetting.” <https://arxiv.org/pdf/2211.12447>, 2022.
- [37] Clark, Bryan. “Quantum Fourier Transform.” University of Illinois Urbana-Champaign, 2022. <https://courses.physics.illinois.edu/phys498cmp/sp2022/QC/QFT.html>.
- [38] Cleve, Richard. “Classical Lower Bound for Simon’s Problem.” University of Waterloo, 2011. <https://cs.uwaterloo.ca/~cleve/courses/F11CS667/SimonClassicalLB.pdf>.
- [39] Cleve, Richard, Artur Ekert, Chiara Macchiavello, and Michele Mosca. “Quantum Algorithms Revisited.” *Proceedings of the Royal Society of London A* 454 (1998): 339–54. <https://doi.org/10.1098/rspa.1998.0164>.

- [40] Cohen, Danny. “On Holy Wars and a Plea for Peace.” *IEEE Computer* 14, no. 10 (1981): 49–54. <https://gwern.net/doc/cs/algorithm/1981-cohen.pdf>.
- [41] Conrad, Keith. “Tensor Products.” <https://kconrad.math.uconn.edu/blurbs/linmultialg/tensorprod.pdf>.
- [42] Cycling ’74. “What Is Max?” <https://cycling74.com/products/max>.
- [43] Daniels, Greg, Brent Forrester, and Ricky Gervais. “A.A.R.M.” (*The Office*, Season 9, Episode 17). <https://www.imdb.com/title/tt2669740>, 2013.
- [44] Dargan, James. “Top 63 Quantum Computer Simulators for 2024.” <https://thequantuminsider.com/2022/06/14/top-63-quantum-computer-simulators-for-2022/>.
- [45] De Morgan, Augustus. *A Treatise on the Calculus of Functions*, 1836.
- [46] Desef, Benjamin. “Yquant.” <https://github.com/projekter/yquant>.
- [47] de Wolf, Ronald. “Quantum Computing: Lecture Notes.” QuSoft, CWI and University of Amsterdam, 2023. <https://arxiv.org/pdf/1907.09415>.
- [48] Deutsch, David. *The Fabric of Reality*. Viking, 1997.
- [49] Deutsch, David. “Quantum Theory, the Church–Turing Principle and the Universal Quantum Computer.” *Proceedings of the Royal Society A*. 85, no. 400 (1985): 97–117. <https://doi.org/10.1098/rspa.1985.0070>.
- [50] Dirac, Paul. “A New Notation for Quantum Mechanics.” In *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 35, 416–18, 1939. <https://doi.org/10.1017/S0305004100021162>.
- [51] Dobrijevic, Daisy. “How Hot Is the Sun?” <https://www.space.com/17137-how-hot-is-the-sun.html>, 2023.
- [52] Duan, Lu-Ming, and Guang-Can Guo. “Probabilistic Cloning and Identification of Linearly Independent Quantum States.” <https://arxiv.org/pdf/quant-ph/9804064>, 1997.
- [53] Duan, Lu-Ming, and Guang-Can Guo. “Two Non-Orthogonal States Can Be Cloned by a Unitary-Reduction Process.” <https://arxiv.org/pdf/quant-ph/9704020>, 1987.
- [54] Dunin, Elonka. “Famous Unsolved Codes and Ciphers.” <https://elonka.com/UnsolvedCodes.html>, 2023.
- [55] Einstein, Albert, Boris Podolsky, and Nathan Rosen. “Can Quantum Mechanical Description of Physical Reality Be Considered Complete?” *Physical Review* 47 (1935): 777–80. <https://doi.org/10.1103/PhysRev.47.777>.
- [56] Etim, Inemesit. “10 Quantum Games That Can Help You Learn the Field of Quantum Computing.” <https://quantumzeitgeist.com/10-quantum-games-that-can-help-you-learn-the-field-of-quantum-computing/>, 2022.
- [57] Euler, Leonhard. *Elements of Algebra*. Longman, Rees, Orme, and Co., 1765. 2015 update of 1828 translation by John Hewlett of the

- original *Vollständige Anleitung zur Algebra*. https://ia800308.us.archive.org/2/items/ElementsOfAlgebraLeonhardEuler2015/ElementsOfAlgebra_LeonhardEuler_Edition2015.pdf.
- [58] Everett III, Hugh. ““Relative State” Formulation of Quantum Mechanics.” *Reviews of Modern Physics* 29, no. 3 (1957): 454–62. <https://typeset.io/papers/relative-state-formulation-of-quantum-mechanics-3crdqquh8o>.
 - [59] Faye, Jan. “Copenhagen Interpretation of Quantum Mechanics.” In Edward N. Zalta (ed.), *The Stanford Encyclopedia of Philosophy* (Winter 2019 Edition). <https://plato.stanford.edu/archives/win2019/entries/qm-copenhagen/>.
 - [60] Feldman, Joel. “Trig Identities—Cosine Law and Addition Formulae.” <https://personal.math.ubc.ca/~feldman/m100/trigId.pdf>, 2000.
 - [61] Fermilab. “Physics Questions People Ask Fermilab.” <https://www.fnal.gov/pub/science/inquiring/questions/atoms.html>, 2014.
 - [62] Feynman, Richard P. “From a Letter to Ms. J. M. Szabados of Victoria, Australia.” https://en.wikiquote.org/wiki/Richard_Feynman, 1965.
 - [63] Fisher, Trevor, Derek Funk, and Rachel Sams. “The Birthday Problem and Generalizations.” https://d31kydh6n6r5j5.cloudfront.net/uploads/sites/66/2019/04/birthday_comps.pdf, 2013.
 - [64] Florio, G., and D. Picca. “Quantum Implementation of Elementary Arithmetic Operations.” <https://arxiv.org/pdf/quant-ph/0403048>, 2004.
 - [65] Freeze, Brennan T., Aundre Barras, Paris Osuch, Soren Sevier Richenberg, and Suzanne Rivoire. “QC.py: Quantum Computing Simulation and Visualization Suite.” In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education*, vol. 2, 2023. <https://doi.org/10.1145/3545947.3576334>.
 - [66] Gabel, Robert A., and Richard A. Roberts. *Signals and Linear Systems* (2nd ed.). John Wiley & Sons, 1980.
 - [67] Gagliardoni, Tommaso. “Quantum Attack Resource Estimate: Using Shor’s Algorithm to Break RSA vs DH/DSA vs ECC.” <https://research.kudelskisecurity.com/2021/08/24/quantum-attack-resource-estimate-using-shors-algorithm-to-break-rsa-vs-dh-dsa-vs-ecc/>, 2021.
 - [68] Galilei, Galileo. *Il Saggiatore*. 1623. Quoted at <https://mathshistory.st-andrews.ac.uk/Biographies/Galileo/quotations/>.
 - [69] Gambetta, Jay. “The Hardware and Software for the Era of Quantum Utility Is Here.” IBM Quantum Research Blog, December 2023. <https://www.ibm.com/quantum/blog/quantum-roadmap-2033>.
 - [70] Gauss, Carl Friedrich. *Theoria Residiorum Biquadraticorum, Commentario Secunda*. Dieterich, 1832. Quoted in Robert Edouard Moritz. *Memorabilia Mathematica* (Macmillan, 1914). <https://www.gutenberg.org/cache/epub/44730/pg44730-images.html>.

- [71] Gharibian, Sevag. “Introduction to Quantum Computation.” Paderborn University, 2021. https://groups.uni-paderborn.de/jg-qi/courses/UPB_INTRO_QUANTUM/S2021/notes/IQC_Masterfile.pdf.
- [72] Gide, André. *The Counterfeiters: A Novel*. Knopf, 1927.
- [73] Glassner, Andrew. “An Introduction to the Fourier Transform.” In *SIGGRAPH Courses '24*, 2024. <https://dl.acm.org/doi/10.1145/3664475.3664537>.
- [74] Glassner, Andrew. “Quantum Computing, Part 1.” <https://www.glassner.com/wp-content/uploads/2014/04/CG-CGA-PDF-01-07-Quantum-Computing-1-July01.pdf>, 2001.
- [75] Glassner, Andrew. “Quantum Computing, Part 2.” <https://www.glassner.com/wp-content/uploads/2014/04/CG-CGA-PDF-01-09-Quantum-Computing-2-Sept01.pdf>, 2001.
- [76] Glassner, Andrew. “Quantum Computing, Part 3.” <https://www.glassner.com/wp-content/uploads/2014/04/CG-CGA-PDF-01-11-Quantum-Computing-3-Nov01.pdf>, 2001.
- [77] Gleick, James. *Chaos: Making a New Science*. Viking, 1987.
- [78] Google Quantum AI. “Cirq.” <https://quantumai.google/cirq>.
- [79] Gowers, Timothy. “How to Lose Your Fear of Tensor Products.” <https://www.dpmms.cam.ac.uk/~wtg10/tensors3.html>.
- [80] Graham, Ian. “What Is the Geological Time Scale?” <https://australian.museum/learn/australia-over-time/evolving-landscape/the-geological-time-scale/>, 2020.
- [81] Griffiths, David J., and Darrell F. Schroeter. *Introduction to Quantum Mechanics* (3rd ed.). Cambridge University Press, 2018.
- [82] Grover, Lov K. “A Fast Quantum Mechanical Algorithm for Database Search.” <https://arxiv.org/pdf/quant-ph/9605043>, 1996.
- [83] Grover, Lov K. “Quantum Computers Can Search Arbitrarily Large Databases by a Single Query.” *Physical Review Letters* 79 (1997): 4709–12. <https://arxiv.org/pdf/quant-ph/9706005.pdf>.
- [84] Guinness World Records. “Heaviest Watermelon.” <https://www.guinnessworldrecords.com/world-records/heaviest-watermelon>.
- [85] Haber, Howard E. “Coordinates, Matrix Elements and Changes of Basis.” University of California, Santa Cruz, 2006. <http://scipp.ucsc.edu/~haber/archives/physics116A06/basis.pdf>.
- [86] Haener, Thomas, Mathias Soeken, Martin Roetteler, and Krysta M. Svore. “Quantum Circuits for Floating-point Arithmetic.” <https://arxiv.org/pdf/1807.02023>, 2018.
- [87] Haldane, J.B.S. *Possible Worlds and Other Essays*. Chatto & Windus, 1927.
- [88] Hamilton, Kirk. “‘Soul Vaccination’ by Tower of Power.” *Strong Songs* podcast, 2023. <https://strongsongsodcast.com>.

- [89] Harper, David, and L.M. Stockman. “The History of Fortran.” <https://www.obliquity.com/computer/fortran/history.html>, 2013.
- [90] Hatfield, Gary. “René Descartes.” <https://plato.stanford.edu/entries/descartes/>, 2023.
- [91] Hoexum, Eline Sophie. “Revisiting the Proof of the Complexity of the Sudoku Puzzle.” Master’s thesis, Rijksuniversiteit Groningen, June 2020. https://fse.studenttheses.ub.rug.nl/22745/1/bMATH_2020_HoexumES.pdf.pdf.
- [92] Hughes, John. Personal communication, 2023.
- [93] Hunt, Leigh. As quoted in *The Farmer’s Wife*, 36, 1933. https://en.wikiquote.org/wiki/Leigh_Hunt.
- [94] Hunziker, Markus, David A. Meyer, Jihun Park, James Pommersheim, and Mitch Rothstein. “The Geometry of Quantum Learning.” *Quantum Information Processing* 9, no. 3 (2003): 321–41. <https://arxiv.org/abs/quant-ph/0309059>.
- [95] IBM. “IBM Quantum Platform.” <https://quantum.ibm.com>.
- [96] IBM. “Qiskit Documentation.” <https://www.ibm.com/quantum/qiskit>.
- [97] IBM. “Quantum Fourier Transform.” <https://github.com/Qiskit/textbook/blob/main/notebooks/ch-algorithms/quantum-fourier-transform.ipynb>, 2022.
- [98] IBM. “Single Qubit Gates.” <https://qiskit.org/textbook/ch-states/single-qubit-gates.html>, 2023.
- [99] IBM Quantum Learning. “Grover’s Algorithm.” <https://learning.quantum.ibm.com/tutorial/grovers-algorithm>.
- [100] Jacobson, Scott. “Crows Encounters of the Bird Kind” (*Bob’s Burgers*, Season 13, Episode 17). <https://www.imdb.com/title/tt27003633>, 2023.
- [101] Jaques, Samuel, and Arthur G. Rattew. “QRAM: A Survey and Critique.” <https://arxiv.org/pdf/2305.10310.pdf>, 2023.
- [102] Johansson, J.R., P.D. Nation, and Franco Nori. “QuTiP 2: A Python Framework for the Dynamics of Open Quantum Systems.” *Computer Physics Communications* 184, no. 4 (2013): 1234–40. <https://doi.org/10.1016/j.cpc.2012.11.019>.
- [103] Johansson, J.R., P.D. Nation, and Franco Nori. “QuTiP: An Open-Source Python Framework for the Dynamics of Open Quantum Systems.” *Computer Physics Communications* 183, no. 8 (2012): 1760–72. <https://doi.org/10.1016/j.cpc.2012.02.021>.
- [104] Johnston, Eric. “An Exploratory Study in Quantum Acceleration of Ray Tracing.” <https://www.machinelevel.com/qc/doc/Quantum%20Ray%20Tracing.pdf>, 2015.
- [105] Jordan, Louis, and Billy Austin. “Is You Is or Is You Ain’t My Baby?” <https://open.spotify.com/track/0IzMBujKJ24egd9HIK3OhG?si=a2a3cd66a3d54515>, 1943.

- [106] Jordan, Stephen. “Quantum Algorithm Zoo.” <https://quantumalgorithmzoo.org>, 2025.
- [107] Joyce, Alex. “Why Right-Hand Rule for Mazes Works.” <https://www.nytimes.com/1989/09/06/opinion/l-why-right-hand-rule-for-mazes-works-075389.html>, 1989.
- [108] Juster, Norton. *The Phantom Tollbooth*. Epstein & Carroll, 1961.
- [109] Kay, Alastair (DaftWullie). “How Do I Show That a Two-Qubit State Is an Entangled State?” <https://quantumcomputing.stackexchange.com/questions/2263/how-do-i-show-that-a-two-qubit-state-is-an-entangled-state>, 2018.
- [110] Kay, Alastair. “Tutorial on the Quantikz Package.” <https://arxiv.org/pdf/1809.03842>, 2023.
- [111] Kay, Alastair (DaftWullie). “Understanding Oracles for Simon’s Algorithm.” <https://quantumcomputing.stackexchange.com/questions/32360/understanding-oracles-for-simons-algorithm>, 2023.
- [112] Kernighan, Brian W., and Dennis M. Ritchie. *The C Programming Language*. Pearson, 1978.
- [113] Khan Academy. “Intro to the Trigonometric Ratios.” <https://www.khanacademy.org/math/geometry-home/right-triangles-topic/intro-to-the-trig-ratios-geo/v/basic-trigonometry>, 2011.
- [114] Kun, Jeremy. “How to Conquer Tensorphobia.” <https://jeremykun.com/2014/01/17/how-to-conquer-tensorphobia/>, 2014.
- [115] Kun, Jeremy. “Tensorphobia and the Outer Product.” <https://jeremykun.com/2016/03/28/tensorphobia-outer-product/>, 2014.
- [116] Kuttler, Ken. “Multiplication of Matrices.” [https://math.libretexts.org/Bookshelves/Linear_Algebra/A_First_Course_in_Linear_Algebra_\(Kuttler\)/02%3A_Matrices/2.02%3A_Multiplication_of_Matrices](https://math.libretexts.org/Bookshelves/Linear_Algebra/A_First_Course_in_Linear_Algebra_(Kuttler)/02%3A_Matrices/2.02%3A_Multiplication_of_Matrices), 2023.
- [117] Kuttler, Kenneth. *Linear Algebra, Theory and Applications*. 2012. <https://resources.saylor.org/wwwresources/archived/site/wp-content/uploads/2012/02/Linear-Algebra-Kuttler-1-30-11-OTC.pdf>.
- [118] Kwan, Dan, and Daniel Scheinert. *Everything Everywhere All at Once*. <https://www.imdb.com/title/tt6710474>, 2022.
- [119] Lanese, Nicoletta. “Short-Necked Giraffe Relative Discovered in China. It Used Its Helmet Head to Bash Rivals.” <https://www.live-science.com/extinct-giraffe-relative-fossils>, 2022.
- [120] Lankham, Isaiah, Bruno Nachtergaele, and Anne Schilling. “9.1: Inner Products.” [https://math.libretexts.org/Bookshelves/Linear_Algebra/Book%3A_Linear_Algebra_\(Schilling_Nachtergaele_and_Lankham\)/09%3A_Inner_product_spaces/9.01%3A_Inner_Products](https://math.libretexts.org/Bookshelves/Linear_Algebra/Book%3A_Linear_Algebra_(Schilling_Nachtergaele_and_Lankham)/09%3A_Inner_product_spaces/9.01%3A_Inner_Products).
- [121] LEGO. “MINDSTORMS Reference.” <https://makecode.mindstorms.com/reference>, 2018.

- [122] Leiber, Jerry, and Mike Stoller. “Searchin’.” <https://genius.com/The-coasters-searchin-lyrics>, 1957.
- [123] Lennon, John, and Paul McCartney. “Do You Want to Know a Secret?” <https://music.youtube.com/watch?v=uRQ7ecvU56k>, 1964.
- [124] Li, Lvzhou, Jingquan Luo, and Yongzhen Xu. “Playing Mastermind on Quantum Computers.” <https://arxiv.org/pdf/2207.09356>, 2023.
- [125] LibreTexts. “1.5: Composition of Functions.” In *Precalculus 2e* (OpenStax). [https://math.libretexts.org/Bookshelves/Precalculus/Precalculus_2e_\(OpenStax\)/01%3A_Functions/1.05%3A_Composition_of_Functions](https://math.libretexts.org/Bookshelves/Precalculus/Precalculus_2e_(OpenStax)/01%3A_Functions/1.05%3A_Composition_of_Functions).
- [126] Lincoln, Don. “Quantum Foam, Virtual Particles, and Other Curiosities.” <https://www.pbs.org/wgbh/nova/article/quantum-foam-virtual-particles-and-other-curiosities>, 2012.
- [127] Livingston, Morgan, and Richard Liu. “Games Round Up: Quantum Computing.” <https://www.wilsoncenter.org/blog-post/games-round-quantum-computing>, 2022.
- [128] Lu, Xi, and Hongwei Lin. “A Framework for Quantum Ray Tracing.” <https://arxiv.org/pdf/2203.15451>, 2022.
- [129] Ludwig, Howard. “Why Is the Set of All Integers Denoted by $\mathbb{Z}$?” <https://www.quora.com/Why-is-the-set-of-all-integers-denoted-by-mathbb-Z>, 2019.
- [130] Luff, Christine. “How Long Does It Take to Run a Marathon?” <https://www.verywellfit.com/how-long-does-it-take-to-run-a-marathon-2911423>, 2024.
- [131] Martinez, Josu Etxezarreta. “How to Construct Matrix of Regular and ‘Flipped’ 2-qubit CNOT?” <https://quantumcomputing.stackexchange.com/questions/5179/how-to-construct-matrix-of-regular-and-flipped-2-qubit-cnot>, 2019.
- [132] Mathoma. “Geometric Algebra in 2D—Two Reflections Is a Rotation.” <https://www.youtube.com/watch?v=Hy2gdbbrJZ8>, 2016.
- [133] Matsos, Helen. “How Animals Sense Earth’s Magnetic Field.” <https://phys.org/news/2020-05/animals-earth-magnetic-field.html>, 2020.
- [134] McAllister, Willy. “Basic Electrical Quantities: Current, Voltage, Power.” <https://www.khanacademy.org/science/physics/circuits-topic/circuits-resistance/a/ee-voltage-and-current>.
- [135] Merino, Orlando. “A Short History of Complex Numbers.” <https://www.math.uri.edu/~merino/spring06/mth562/ShortHistoryComplexNumbers2006.pdf>, 2006.
- [136] Mermin, N. David. “Could Feynman Have Said This?” *Physics Today* 57, no. 5 (2004): 10–11. <https://doi.org/10.1063/1.1768652>.
- [137] Mermin, N. David. *Quantum Computer Science: An Introduction*. Cambridge University Press, 2007.

- [138] Merriam-Webster. “Black Box.” <https://www.merriam-webster.com/dictionary/black%20box>.
- [139] Merriam-Webster. “Oracle.” <https://www.merriam-webster.com/dictionary/oracle>.
- [140] Microsoft Azure Quantum Documentation. “Dirac Notation in Quantum Computing.” <https://learn.microsoft.com/en-us/azure/quantum/concepts-dirac-notation>, 2025.
- [141] Microsoft Azure Quantum Documentation. “Understanding Quantum Oracles.” <https://learn.microsoft.com/en-us/azure/quantum/concepts-oracles>, 2024.
- [142] Miller-Bakewell, Hector, and John van de Wetering. “The ZX-calculus.” <https://zxcalculus.com>.
- [143] Nahin, Paul J. *Dr. Euler’s Fabulous Formula: Cures Many Mathematical Ills*. Princeton University Press, 2017.
- [144] Najera, Jesus. “The History of Euler’s Number (e).” <https://www.cantorsparadise.org/the-history-of-eulers-number-e-8c982994a39b/>, 2020.
- [145] Nakanishi, Ken M., Takahiko Satoh, and Synge Todo. “Quantum-Gate Decomposer.” <https://arxiv.org/pdf/2109.13223>, 2021.
- [146] Nielsen, Michael A., and Isaac L. Chuang. *Quantum Computation and Quantum Information* (10th anniv. ed.). Cambridge University Press, 2011.
- [147] Neven, Hartmut. “Meet Willow, Our State-of-the-Art Quantum Chip.” <https://blog.google/technology/research/google-willow-quantum-chip/>, 2024.
- [148] Nicholas, Jackie, and Peggy Adamson. “Introduction to Trigonometric Functions.” <https://www.sydney.edu.au/content/dam/students/documents/mathematics-learning-centre/introduction-trigonometric-functions.pdf>, 1998.
- [149] NIST. “NIST Releases First 3 Finalized Post-Quantum Encryption Standards.” <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>, 2024.
- [150] Nykamp, Duane Q. “Linear Transformations.” https://mathinsight.org/linear_transformation_definition_euclidean.
- [151] O’Connor, J.J., and E.F. Robertson. “Jean Robert Argand.” <https://mathshistory.st-andrews.ac.uk/Biographies/Argand/>, 2019.
- [152] Oppenheim, Alan V., and Ronald W. Schaffer. *Digital Signal Processing*. Prentice-Hall, 1975.
- [153] Osgood, Brad. “Lecture Notes for EE 261: The Fourier Transform and Its Applications.” <https://see.stanford.edu/materials/lsoftae261/book-fall-07.pdf>, 2007.
- [154] Padavic-Callaghan, Karmela. “Quantum Computers Have Finally Arrived, But Will They Ever Be Useful?” *New Scientist*, 2025. <https://www.newscientist.com/article/2467128-quantum-computers-have-finally-arrived-but-will-they-ever-be-useful/>.

- [155] Padavic-Callaghan, Karmela. “Quantum ‘Schrödinger’s Cat’ Survives for a Stunning 23 Minutes.” *New Scientist*, 2024. <https://www.newscientist.com/article/2453356-quantum-schrodingers-cat-survives-for-a-stunning-23-minutes/>.
- [156] Pais, A. “Einstein and the Quantum Theory.” *Reviews of Modern Physics* 51 (1979): 863–914. https://eclass.aegean.gr/modules/document/file.php/511165/quantum_mechanics/einstein_quantum.pais%281979%29.pdf.
- [157] Pevzner, Vadim, and Karl Hess. “Quantum Ray Tracing: A New Approach to Quantum Transport in Mesoscopic Systems.” In K. Hess, J.P. Leburton, and U. Ravaioli (eds.), *Computational Electronics*, 227–30. The Springer International Series in Engineering and Computer Science, vol. 113. Springer, 1991. https://doi.org/10.1007/978-1-4757-2124-9_45.
- [158] Pirandola, Stefano, Jens Eisert, Christian Weedbrook, Akira Furusawa, and Samuel L. Braunstein. “Advances in Quantum Teleportation.” *Nature Photonics* 9 (2015): 641–52. <https://arxiv.org/pdf/1505.07831>.
- [159] Pomeroy, Ross. “Did Your Computer Crash? It Might Have Been a Cosmic Ray.” <https://bigthink.com/hard-science/cosmic-rays-computer-crash/>, 2022.
- [160] Preskill, John. “Quantum Computing 40 Years Later.” <https://arxiv.org/pdf/2106.10522.pdf>, 2021.
- [161] Prokopenya, Alexander. “Quantum Circuit Implementing Grover’s Search Algorithm.” <https://demonstrations.wolfram.com/QuantumCircuitImplementingGroversSearchAlgorithm/#popup1>, 2011.
- [162] ProofWiki. “Transpose of Matrix Product.” https://proofwiki.org/wiki/Transpose_of_Matrix_Product, 2024.
- [163] Puzzling World. “The Great Maze.” <https://www.puzzlingworld.co.nz/attractions/the-great-maze>.
- [164] Qiskit authors. “Qiskit.” <https://pypi.org/project/qiskit/>.
- [165] Quantiki. “Bell State.” <https://quantiki.org/wiki/bell-state>, 2015.
- [166] Quantiki. “List of QC Simulators.” <https://quantiki.org/wiki/list-qc-simulators>, 2024.
- [167] Quantum Open Source Foundation. “Open-Source Quantum Software Projects.” <https://github.com/qosf/awesome-quantum-software>, 2025.
- [168] Quantum Open Source Foundation. “Public Repository.” <https://github.com/qosf>, 2025.
- [169] QuTiP developers and contributors. “QuTiP: Quantum Toolbox in Python.” <https://qutip.org>, 2024.
- [170] Rauch, Earl Mac. *The Adventures of Buckaroo Banzai Across the 8th Dimension*. <https://www.imdb.com/title/tt0086856>, 1984.

- [171] Reany, P. “The No-Cloning Theorem Made Easy.” <https://advancedmath.org/Physics/Quantum/QM1/No-Cloning-Theorem.pdf>, 2023.
- [172] Richman, Lynne. “The History and Evolution of Numbers.” <https://sites.math.rutgers.edu/~zeilberg/math436/projects/RichmanP.pdf>, 2017.
- [173] Robbins, Tom. *Even Cowgirls Get the Blues*. Houghton Mifflin, 1976.
- [174] Roberts, Molly. “Numbers With Cool Names: Amicable, Social, Friendly.” <https://tomrocksmaths.com/2023/05/10/numbers-with-cool-names-amicable-sociable-friendly/>, 2023.
- [175] Robinson, Andrew. “Did Einstein Really Say That?” *Nature* 557 (2018). <https://www.nature.com/articles/d41586-018-05004-4>.
- [176] Roddenberry, Gene, and Jerome Bixby. “Mirror, Mirror” (*Star Trek*, Season 2, Episode 4). <https://www.imdb.com/title/tt0708438>, 1970.
- [177] Roffe, Joschka. “Quantum Error Correction: An Introductory Guide.” *Contemporary Physics* 60, no. 3 (2019): 226–45. <https://arxiv.org/pdf/1907.11157>.
- [178] Rogers, Leo. “The History of Negative Numbers.” <https://nrich.maths.org/articles/history-negative-numbers>, 2011.
- [179] The Royal Swedish Academy. “Nobel Prize in Physics 2022: Popular Information.” <https://www.nobelprize.org/prizes/physics/2022/popular-information/>, 2023.
- [180] Rui, Pinshu, Wen Zhangx, Yanlin Liao, and Ziyun Zhang. “Part Probabilistic Cloning of Linearly Dependent States.” <https://arxiv.org/pdf/1603.07036>, 2016.
- [181] Running Level. “1k Run Times.” <https://runninglevel.com/running-times/1k-times>.
- [182] Salton, Grant, Daniel Simon, and Cedric Lin. “Exploring Simon’s Algorithm With Daniel Simon.” <https://aws.amazon.com/blogs/quantum-computing/simons-algorithm/>, 2021.
- [183] Santos, Luís Paulo, Thomas Bashford-Rogers, João Barbosa, and Paul Navrátil. “Towards Quantum Ray Tracing.” <https://arxiv.org/pdf/2204.12797>, 2022.
- [184] Scale of Universe. “How Big Is the Smallest Object Visible to the Naked Eye?” <https://scaleofuniverse.com/en/universe/smallest-object-visible-to-the-naked-eye>, 2023.
- [185] Scholten, Travis L., Bryan Eastin, and Steven Flammia. “Qcircuit—Macros to Generate Quantum Circuits.” <https://www.ctan.org/pkg/qcircuit>, 2018.
- [186] Schuch, Norbert. “How to Construct a Multi-qubit Controlled-Z From Elementary Gates?” <https://quantumcomputing.stackexchange.com/questions/4078/how-to-construct-a-multi-qubit-controlled-z-from-elementary-gates>, 2018.

- [187] Schuch, Norbert, and Jens Siewert. “Programmable Networks for Quantum Algorithms.” *Physical Review Letters* 91, no. 2 (2003): 027902. <https://arxiv.org/pdf/quant-ph/0303063>.
- [188] Schwichtenberg, Jakob. *No-Nonsense Quantum Mechanics*. No Nonsense Books, 2020.
- [189] Schwitzgebel, Eric. “How to Wrap Your Head around the Most Mind-bending Theories of Reality.” *New Scientist*, 2024. <https://www.newscientist.com/article/mg26134833-100-how-to-wrap-your-head-around-the-most-mind-bending-theories-of-reality/>.
- [190] Scratch Consortium. “Scratch Reference.” <https://scratch.mit.edu>.
- [191] Seneca the Younger. “Moral and Political Essays.” https://en.wikiquote.org/wiki/Seneca_the_Younger, 40.
- [192] Shakespeare, William. *The Tragedy of Coriolanus*. 1605. <https://www.folger.edu/explore/shakespeares-works/coriolanus/read/>.
- [193] Shannon, C.E. “A Mathematical Theory of Communication.” *Bell System Technical Journal* 27, no. 3 (1948): 379–423. <https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>.
- [194] Shor, Peter. “18.435/2.111 POVM Lecture.” Massachusetts Institute of Technology, 2008. <https://math.mit.edu/~shor/18.435/POVM-lecture.pdf>.
- [195] Shor, Peter. “Algorithms for Quantum Computation: Discrete Logarithms and Factoring.” In *Proceedings, 35th Annual Symposium on Foundations of Computer Science*, 124–34. IEEE Computer Society Press, 1994. <https://math.mit.edu/~shor/papers/algsfqcdlf.pdf>.
- [196] Shor, Peter. “Lecture Notes for 8.360/18.435 Quantum Computation.” Massachusetts Institute of Technology, 2022. <https://math.mit.edu/~shor/435-LN/>.
- [197] SideFX. “Houdini.” <https://www.sidefx.com/products/houdini/>, 2025.
- [198] Simha, Rahul. “Introduction to Quantum Computing.” <https://www2.seas.gwu.edu/~simhaweb/quantum/modules.html>, 2024.
- [199] Simmons, Gustavus J. “RSA Encryption.” <https://www.britannica.com/topic/RSA-encryption>, 2022.
- [200] Simon, Daniel R. “On the Power of Quantum Computation.” *SIAM Journal on Computing* 26, no. 5 (1997): 1474–83. <https://doi.org/10.1137/S0097539796298637>.
- [201] Sloane, N. J. A. “The On-Line Encyclopedia of Integer Sequences.” <https://oeis.org/A000040>, 2025.
- [202] Smith, Kurt. “Descartes’ Theory of Ideas.” <https://plato.stanford.edu/Archives/fall2013/entries/descartes-ideas/>, 2013.
- [203] Solow, Daniel. *How to Read and Do Proofs* (6th ed.). Wiley, 2013.

- [204] Sparkes, Matthew. “Google’s Claim of Quantum Supremacy Has Been Completely Smashed.” *New Scientist*, 2024. <https://www.newscientist.com/article/2437886-googles-claim-of-quantum-supremacy-has-been-completely-smashed/>.
- [205] Sparknotes authors. “The Allegory of the Cave.” <https://www.sparknotes.com/philosophy/republic/themes/>.
- [206] Spivery, William. “Operator.” <https://www.youtube.com/watch?v=WEAtDmTbp4>, 1944.
- [207] Stack Exchange. “All Sites.” <https://stackexchange.com/sites#>.
- [208] Stack Exchange. “Mathematics Stack Exchange.” <https://math.stackexchange.com>.
- [209] Stack Exchange. “Physics Stack Exchange.” <https://physics.stackexchange.com>.
- [210] Stack Exchange. “Quantum Computing Stack Exchange.” <https://quantumcomputing.stackexchange.com>.
- [211] Strang, Gilbert. *Introduction to Linear Algebra* (6th ed.). MIT Press, 2023.
- [212] Strogatz, Steven. “How Infinite Series Reveal the Unity of Mathematics.” <https://www.quantamagazine.org/how-infinite-series-reveal-the-unity-of-mathematics-20220124/>, 2022.
- [213] Susskind, Leonard, and Art Friedman. *Quantum Mechanics: The Theoretical Minimum*. Basic Books, 2014.
- [214] Swift, Jonathan. *Gulliver’s Travels into Several Remote Nations of the World*. Benjamin Motte, 1726. <https://www.gutenberg.org/files/829/829-h/829-h.htm>.
- [215] Szabó, László E. “The Einstein–Podolsky–Rosen Argument and the Bell Inequalities.” <https://arxiv.org/pdf/0712.1318>, 2007.
- [216] Tagore, Rabindranath. *Stray Birds*. Macmillan, 1916. <https://www.gutenberg.org/files/6524/6524-h/6524-h.htm>.
- [217] Tavares, Américo. “Understanding Imaginary Exponents.” <https://math.stackexchange.com/questions/9770/understanding-imaginary-exponents>, 2010.
- [218] Tegmark, Max. “Shut Up and Calculate.” <https://arxiv.org/pdf/0709.4024>, 2007.
- [219] Tomayko, James E. *Computers in Spaceflight: The NASA Experience*. 1988. <https://history.nasa.gov/computers/Source2.html>.
- [220] Treil, Sergei. *Linear Algebra Done Wrong*. 2017. https://www.math.brown.edu/streil/papers/LADW/LADW_2017-09-04.pdf.
- [221] University of Alberta, Centre for Teaching and Learning. “Why Reflect on Your Teaching?” <https://www.ualberta.ca/centre-for-teaching-and-learning/teaching-support/reflection/why-reflect-on-your-teaching/index.html>.

- [222] van de Wetering, John. “ZX-calculus for the Working Quantum Computer Scientist.” <https://arxiv.org/pdf/2012.13966>, 2020.
- [223] van Gogh, Vincent. Letter to Theo van Gogh. <https://vangoghletters.org/vg/letters/let274/letter.html>, 1882.
- [224] Vazirani, Umesh V. “Lecture 3: Hilbert Spaces, Tensor Products, Teleportation.” <https://people.eecs.berkeley.edu/~vazirani/f04quantum/notes/lec3.pdf>, 2004.
- [225] Vazirani, Umesh V. “Lecture 4: Quantum Circuit Model, Solovay–Kitaev Theorem, BQP.” <https://people.eecs.berkeley.edu/~vazirani/f04quantum/notes/lec4.pdf>, 2004.
- [226] Vernimmen, Tim. “Sharks Can Navigate Via Earth’s Magnetic Field, Study Confirms for the First Time.” <https://www.nationalgeographic.com/animals/article/sharks-can-navigate-via-earths-magnetic-field-study-confirms-for-the-first-time>, 2021.
- [227] Waits, Tom. “Foreign Affair.” <https://www.youtube.com/watch?v=-iTLk3gjEec>, 1977.
- [228] Weaver, James, Huang Junye, Jarrod Reilly, and Anastasia Jeffery. “QPong,” 2019. <https://github.com/QPong/QPong>, <https://github.com/QPong/QPong-Unity>, and www.youtube.com/watch?v=a1NZC5rqQD8.
- [229] Weinersmith, Zach. “Saturday Morning Breakfast Cereal.” <https://www.smbc-comics.com/comic/contrived>, 2023.
- [230] Weisberger, Mindy. ““God Plays Dice With the Universe,” Einstein Writes in Letter About His Qualms With Quantum Theory.” <https://www.livescience.com/65697-einstein-letters-quantum-physics.html>, 2019.
- [231] Weisstein, Eric W. “Argand Diagram.” <https://mathworld.wolfram.com/ArgandDiagram.html>.
- [232] Weisstein, Eric W. “Birthday Problem.” <https://mathworld.wolfram.com/BirthdayProblem.html>.
- [233] Weisstein, Eric W. “Bracket.” <https://mathworld.wolfram.com/Bracket.html>.
- [234] Weisstein, Eric W. “Euclidean Algorithm.” <https://mathworld.wolfram.com/EuclideanAlgorithm.html>.
- [235] Weisstein, Eric W. “Euclid’s Theorems.” <https://mathworld.wolfram.com/EuclidsTheorems.html>.
- [236] Weisstein, Eric W. “Exponential Function.” <https://mathworld.wolfram.com/ExponentialFunction.html>.
- [237] Weisstein, Eric W. “Fundamental Theorem of Arithmetic.” <https://mathworld.wolfram.com/FundamentalTheoremofArithmetic.html>.
- [238] Weisstein, Eric W. “Gaussian Elimination.” <https://mathworld.wolfram.com/GaussianElimination.html>.
- [239] Weisstein, Eric W. “Hilbert Space.” <https://mathworld.wolfram.com/HilbertSpace.html>.

- [240] Weisstein, Eric W. “Kronecker Delta.” <https://mathworld.wolfram.com/KroneckerDelta.html>.
- [241] Weisstein, Eric W. “Linear Function.” <https://mathworld.wolfram.com/LinearFunction.html>.
- [242] Weisstein, Eric W. “List.” <https://mathworld.wolfram.com/List.html>.
- [243] Weisstein, Eric W. “Natural Number.” <https://mathworld.wolfram.com/NaturalNumber.html>.
- [244] Weisstein, Eric W. “Operator.” <https://mathworld.wolfram.com/Operator.html>.
- [245] Weisstein, Eric W. “Prime Number.” <https://mathworld.wolfram.com/PrimeNumber.html>.
- [246] Weisstein, Eric W. “Set.” <https://mathworld.wolfram.com/Set.html>.
- [247] Weisstein, Eric W. “Square Root.” <https://mathworld.wolfram.com/SquareRoot.html>.
- [248] Weisstein, Eric W. “Vector Space.” <https://mathworld.wolfram.com/VectorSpace.html>.
- [249] Whitfield, James D., Jun Yang, Weishi Wang, Joshua T. Heath, and Brent Harrison. “Quantum Computing 2022.” <https://arxiv.org/pdf/2201.09877.pdf>, 2022.
- [250] Wikipedia. “Bell State.” https://en.wikipedia.org/wiki/Bell_state.
- [251] Wikipedia. “Bernstein–Vazirani Algorithm.” https://en.wikipedia.org/wiki/Bernstein%E2%80%93Vazirani_algorithm.
- [252] Wikipedia. “Bloch Sphere.” https://en.wikipedia.org/wiki/Bloch_sphere.
- [253] Wikipedia. “Bohr Radius.” https://en.wikipedia.org/wiki/Bohr_radius.
- [254] Wikipedia. “Born Rule.” https://en.wikipedia.org/wiki/Born_rule.
- [255] Wikipedia. “Bra–ket Notation.” https://en.wikipedia.org/wiki/Bra%E2%80%93ket_notation.
- [256] Wikipedia. “Class (Computer Programming).” [https://en.wikipedia.org/wiki/Class_\(computer_programming\)](https://en.wikipedia.org/wiki/Class_(computer_programming)).
- [257] Wikipedia. “Density Matrix.” https://en.wikipedia.org/wiki/Density_matrix.
- [258] Wikipedia. “Deutsch–Jozsa Algorithm.” https://en.wikipedia.org/wiki/Deutsch-Jozsa_algorithm.
- [259] Wikipedia. “ e (Mathematical Constant).” [https://en.wikipedia.org/wiki/E_\(mathematical_constant\)](https://en.wikipedia.org/wiki/E_(mathematical_constant)).
- [260] Wikipedia. “Euler’s Formula.” https://en.wikipedia.org/wiki/Euler%27s_formula.
- [261] Wikipedia. “Felix Bloch.” https://en.wikipedia.org/wiki/Felix_Bloch.
- [262] Wikipedia. “Fortran.” <https://en.wikipedia.org/wiki/Fortran>.
- [263] Wikipedia. “Fredkin Gate.” https://en.wikipedia.org/wiki/Fredkin_gate.

- [264] Wikipedia. "Gaussian Elimination." https://en.wikipedia.org/wiki/Gaussian_elimination.
- [265] Wikipedia. "Greatest Common Divisor." https://en.wikipedia.org/wiki/Greatest_common_divisor.
- [266] Wikipedia. "'Hello, World!' Program." https://en.wikipedia.org/wiki/%22Hello,_World!%22_program.
- [267] Wikipedia. "Invertible Matrix." https://en.wikipedia.org/wiki/Invertible_matrix.
- [268] Wikipedia. "Jaques Hadamard." https://en.wikipedia.org/wiki/Jacques_Hadamard.
- [269] Wikipedia. "John von Neumann." https://en.wikipedia.org/wiki/John_von_Neumann.
- [270] Wikipedia. "Key Size." https://en.wikipedia.org/wiki/Key_size.
- [271] Wikipedia. "Matrix Multiplication." https://en.wikipedia.org/wiki/Matrix_multiplication.
- [272] Wikipedia. "Natural Number." https://en.wikipedia.org/wiki/Natural_number.
- [273] Wikipedia. "Noisy Intermediate-Scale Quantum Era." https://en.wikipedia.org/wiki/Noisy_intermediate-scale_quantum_era.
- [274] Wikipedia. "NP (Complexity)." [https://en.wikipedia.org/wiki/NP_\(complexity\)](https://en.wikipedia.org/wiki/NP_(complexity)).
- [275] Wikipedia. "Oracle Machine." https://en.wikipedia.org/wiki/Oracle_machine.
- [276] Wikipedia. "POVM." <https://en.wikipedia.org/wiki/POVM>.
- [277] Wikipedia. "Quantum Fourier Transform." https://en.wikipedia.org/wiki/Quantum_Fourier_transform.
- [278] Wikipedia. "Superdense Coding." https://en.wikipedia.org/wiki/Superdense_coding.
- [279] Wikipedia. "Toffoli Gate." https://en.wikipedia.org/wiki/Toffoli_gate.
- [280] Wikipedia. "Triple Modular Redundancy." https://en.wikipedia.org/wiki/Triple_modular_redundancy.
- [281] Wikipedia. "Unitary Matrix." https://en.wikipedia.org/wiki/Unitary_matrix.
- [282] Wikipedia. "Vector (Mathematics and Physics)." [https://en.wikipedia.org/wiki/Vector_\(mathematics_and_physics\)](https://en.wikipedia.org/wiki/Vector_(mathematics_and_physics)).
- [283] Wikipedia. "Vector Space." http://en.wikipedia.org/wiki/Vector_space.
- [284] Wikipedia. "Wolfgang Pauli." https://en.wikipedia.org/wiki/Wolfgang_Pauli.
- [285] Wikipedia. "Zero to the Power of Zero." https://en.wikipedia.org/wiki/Zero_to_the_power_of_zero.

- [286] Wilson, Robin. *Euler's Pioneering Equation: The Most Beautiful Theorem in Mathematics*. Oxford University Press, 2018.
- [287] Wolfram. "Wolfram Player." <https://www.wolfram.com/player/>.
- [288] Wootten, James. "Making Games With Quantum Computers." <https://decodoku.medium.com/games-computers-and-quantum-84bfdd2c0fe0>, 2020.
- [289] Yang, Jianhao M. "Probabilistic Quantum Teleportation." <https://arxiv.org/pdf/1601.02501>, 2017.
- [290] Yang, Yu, Igor Kladaric, Maxwell Drimmer, et al. "A Mechanical Qubit." *Science* 386, no. 6723 (2024): 783–88. <https://www.science.org/doi/10.1126/science.adr2464>.
- [291] Yang, Y.A., W.-T. Luo, J.-L. Zhang, S.-Z. Wang, Chang-Ling Zou, T. Xia, and Z.-T. Lu. "Minutes-scale Schrödinger-cat State of Spin-5/2 Atoms." *Nature Photonics* 19 (2024): 89–94. <https://arxiv.org/pdf/2410.09331v1>.
- [292] Young, Peter. "Simon's Algorithm." <https://web.archive.org/web/20231004205621/https://young.physics.ucsc.edu/150/simon.pdf>, 2019.
- [293] Young, Peter. "An Undergraduate Course on Quantum Computing" (4th ed.). University of California, Santa Cruz, 2024. https://bpb-us-e1.wpmucdn.com/sites.ucsc.edu/dist/7/1905/files/2025/03/phys_150_all.pdf.
- [294] Young, Peter. "Using Period Finding to Factor an Integer." <https://web.archive.org/web/20221028100132/https://young.physics.ucsc.edu/150/period.pdf>, 2019.
- [295] Zhang, Naiyuan J., Ron Q. Nguyen, Navketan Batra, et al. "Excitons in the Fractional Quantum Hall Effect." *Nature* 637 (2025): 327–32. <https://doi.org/10.1038/s41586-024-08274-3>.
- [296] Zhang, Yingwen, Antony Orth, Duncan England, and Benjamin Sussman. "Ray Tracing With Quantum Correlated Photons to Image a Three-dimensional Scene." *Physical Review A* 105 (2022): L011701. <https://doi.org/10.1103/PhysRevA.105.L011701>.
- [297] Zhu, Sebastian, William Yue, and Vincent Fan. "Shor's Algorithm and the Period Finding Problem." https://math.mit.edu/research/highschool/primes/materials/2019/December/Fan-Yue-Zhu_Shor_s_Algorithm_and_the_Period_Finding_Problem.pdf, 2019.

INDEX

Note: *Italicized page numbers refer to figures or tables.*

Numbers

- 0-active control line, 311
- 1-active control line, 311

A

- Adams, Douglas, 203
- adjoint, 74, 92–93
- Adleman, Leonard, 317
- algorithms
 - Bernstein–Vazirani, 269–276
 - Deutsch–Jozsa, 251–267
 - Deutsch’s, 227–249
 - Euclidean, 330
 - Grover’s, 295–314
 - hybrid, 126
 - probabilistic, 331
 - Shor’s, 315–335
 - Simon’s, 277–294
- amplitude, 101
 - amplification, 304
 - electronic playing cards
 - metaphor, 16–18
 - estimation, 314
 - outer product, 172–176
 - from projection, 171–179
 - square of, 17
 - squaring magnitude of, 169
- analysis
 - by algebra, 132
 - bidirectional, 137–138
 - by direction, 132–138
 - horizontal, 133–134
 - by matrix elements, 132–133
 - vertical, 134–137
- ancilla, 231
- Apollo spacecraft, 163

- applied mathematics, 316
- arctangent, 36
- Argand, Jean Robert, 35
- Argand diagram, 35
- argument, 13
- arrows, 56, 57, 58, 63
- assembly language, 203
- auxiliary input, 231

B

- balanced function
 - Deutsch–Jozsa algorithm, 265–266
 - Deutsch’s algorithm, 228
- balanced oracle, 242, 252
- Bartlett, Albert, 279
- bases, 54–55
- basis
 - change of, 65–66
 - computational, 54
 - conventional, 54
- basis set, 54
- basis vectors, 54
- Battleship* game, 350
- Bell, John, 190
- Bell states, 190
 - equations, 191
 - measuring, 193–197
 - superposition, 191–192
- Bernstein, Ethan, 270
- Bernstein–Vazirani algorithm, 269–276.
 - See also* quantum algorithms
 - classical solution, 270
 - compact version of circuit, 270
 - expanded diagram for, 271
 - oracle for classical bits, 270–271, 271
 - running, 273
 - simplified oracle, 275
 - simplifying with quantum identity, 274–275

- bidirectional analysis, 133, 137–138
- big-endianness, 180
- birthday problem, 284
- bits, 27, 31, 159
- bitstrings, 49
 - bitstring dot product, 261
- black box, 229
- Blackjack, 21
- Bloch sphere, 81
- block matrix, 144
- bra, 74
- braket notation, 71–79
- bramket, 173
- branch, 126
- Brooks, Mel, 351
- Bulls and Cows* game, 350
- bundle, 254

C

- cardinality, 30
- Cartesian diagram, 35
- Cartesian product, 48, 112
- Cartwright, Jon, 166
- Carver, George Washington, 269
- CCNOT* qugate (*CCX* qugate, *conditional-conditional-NOT* qugate), 153–154
- chevrons, 68
- Chuang, Isaac, 343
- circuit analysis
 - analysis by algebra, 131–138
 - analysis by direction, 132–138
 - analysis by matrix elements, 132–133
 - bidirectional analysis, 137–138
 - after Deutsch–Jozsa algorithm, 271–272
 - horizontal analysis, 133–134
 - probability of k , 326–328
 - quantum Fourier transform, 324–326
 - Shor’s algorithm, 322–331
 - systems, 131–138
 - vertical analysis, 134–137
- circuit diagrams, 124
- Cirq, 344
- class, 199
- classical computers, 5–6, 27, 163
- clone, 138
- closed loop, 382
- closed under addition, 31
- CNOT* qugate *See* *CX* qugate
- code, 126
- coefficient, 55
- collapse, 178
- column matrix, 50, 72
- commutative, 38, 87, 114
- completeness relation, 188
- complex conjugate, 37
- complexity, computational, 234
- complex magnitude, 68
- complex number plane, 34
- complex numbers, 31, 32, 41–48
 - complex conjugate, 37
 - exponential form, 36–37
 - imaginary component/part, 34
 - magnitude, 35
 - multiplication, 40
 - phase, 35
 - polar form, 35
 - real component/part, 34
 - visualization, 34–36
- complex-valued vectors, 50, 66
- complex vectors, 42, 66, 68
- composite numbers, 316
- composition of functions, 87
- computational basis, 54
- computational basis states, 118
- computational complexity, 234
- computers
 - classical, 5–6, 27, 163
 - conventional, 5–6, 27
 - von Neumann machines, 5
- Confucius, 298
- conjugate, 37
- conjugate symmetry, 71
- conjugation, 37–41, 69, 92
- constant function
 - Deutsch–Jozsa algorithm, 264–265
 - Deutsch’s algorithm, 228
- constant oracle, 241–242
- constructive interference, 18.
 - See also* interference
- continued fractions, 333
- continuous processes, 124

- contradiction in proofs, 139
- controlled-SWAP qugate, 153–154
- controlled-*U* qugate, 151–152, 152
- controlled-*X* qugate *See CX* qugate
- controlled-*Z* qugate, 154–155
- control qubit, 142
- conventional basis, 54
- conventional computers, 5–6, 27
- coordinate, 55
- coordinate-free expression, 64
- Copenhagen interpretation, 346
- copier, 142, 146
- coprimes, 330
- cosine double-angle rule, 59
- counting numbers, 30
- cryptography
 - post-quantum, 340–341
 - quantum, 334, 340–341
 - RSA, 317
- CU* qugate, 151–152, 152
- CX* qugate (*conditional/controlled-NOT* qugate, *conditional/controlled-X* qugate), 141–146
 - as copier, 146
 - entangled pairs, 147–151
 - entanglement, 146–150
 - as switch, 142–146
 - symbols, 142
- CX'* qugate, 153, 212, 274
- CZ* qugate, 154–155

D

- DAG (directed acyclic graph), 126
- Daniels, Greg, 277
- database, unstructured, 296
- deferred measurement,
 - principle of, 189
- degenerate matrix, 93
- De Morgan, Augustus, 351
- density matrix, 339–340
- Descartes, Rene, 35
- destructive interference, 18, 106.
 - See also* interference
- deterministic, classical algorithms as, 160
- Deutsch, David, 228, 251
- Deutsch-Jozsa algorithm, 251–267. *See also* quantum algorithms
 - abstract block diagram for, 253

- actual results of, 266
- balanced function, 252, 265–266
- block diagram for, 254
- circuit analysis after, 271–272
- compact block diagram, 254
- constant function, 252, 264–265
- four functions, 252–253
- Hadamard transform, 259–263
- initialization step, 255–257
- postprocessing and measurement, 258–259, 263–264
- querying the oracle, 257–258
- Deutsch's algorithm, 227–249. *See also* quantum algorithms
 - block diagram, 238
 - Deutsch's problem, 228–229
 - four functions, 228, 228–229, 245
 - function implementations, 246
 - improved block diagram, 238
 - initialization step, 239, 239–240
 - oracles, 229–234
 - phase kickback, 243–245
 - postprocessing and measurement, 242, 242–243
 - quantum parallelism, 234–237, 247–248
 - querying the oracle, 240, 240–242
- Deutsch's problem, 228–229
- diagram, circuit, 124
- diffusion stage, Grover's algorithm, 305–307, 312

dimensionality, 30, 50

Dirac notation, 71

Dirac, Paul, 71, 74

directed acyclic graph (DAG), 126

division, 316

dot product, 56–66

- bitstring, 261

- change of basis, 65–66

- defined, 56

- projection, 62–65

E

Einstein, Albert, 159, 196

electronic playing cards, 6–7

- amplitudes, 16–18

- entanglement, 19–24, 20, 22, 24

- illustration of, 6

- electronic playing cards (*continued*)
 - initialization, 10, 10–11, 11
 - interference, 18–19, 19
 - measurement, 11–12
 - operators, 12–16
 - probabilities, 18
 - states, 7–9, 8, 9
 - superposition, 9, 9–10
- elements, 28
- embarrassingly parallel, 349
- encryption, 315
 - overview, 315–316
 - primes and, 316–318
- entangled pairs
 - CX qugate, 147–151
 - electronic playing cards
 - metaphor, 21–24
- entangled state, 148
- entanglement, 1, 19, 146. *See also*
 - superposition
 - Bell states, 190
 - CX qugate, 146–150
 - electronic playing cards
 - metaphor, 19–24, 20, 22, 24
 - measurement and, 190–197
 - types of, 23–24, 24
- EPR paper, 196
- equal superposition, 10
- Euclidean algorithm, 330
- Euler, Leonhard, 32
- Euler’s formula, 36–37
- Euler’s identity, 41
- Euler’s number, 37, 278
- evaluation, 227, 228
- evolution, 125
- exclusive, mutually, 186
- exclusive OR, 231
- experiments, 162
- exponential form, 37
- exponential function, 278
- exponential speedup, 278, 294

F

- $F1$ function, 282
- $F2$ function, 281
- factor, 317
- Feynman, Richard, 337

- finite set, 29
- floating-point numbers, 31
- Forrester, Brent, 277
- FORTTRAN programming
 - language, 203
- Fourier transform, 322
- Fredkin qugate, 153–154
- Friedman, Art, 49
- functions
 - balanced, 228
 - composition of, 87
 - constant, 228
 - Deutsch–Jozsa algorithm, 265–266
 - exponential, 278
 - one-to-one, 282
 - one-way, 317
 - query of, 228
- fundamental theorem of
 - arithmetic, 317

G

- Galilei, Galileo, 1
- games, 350
- gate
 - classical, 100–101
 - quantum, 100
- Gauss, Carl Friedrich, 32, 290
- Gaussian elimination, 290, 293
- gcd (greatest common divisor), 330
- Gervais, Ricky, 277
- Gide, André, 109
- Gleick, James, 51
- global phase, 197–199
- graph, 349, 126
- greatest common divisor (gcd), 330
- Grover, Lov, 295
- Grover iteration, 306–307, 307
- Grover operation, 306–307, 307
- Grover’s algorithm, 295–314. *See also*
 - quantum algorithms
 - amplitude estimation, 314
 - diffusion stage, 305–307, 312
 - example of, 310–313
 - ideal number of steps, 308–310
 - initialization step, 310
 - marking stage, 301–305
 - observations, 313–314

- oracle, 308–310
- oracle complexity, 313
- overshoot, 308
- quadratic speedup, 313–314
- reflections, 298–300
- symbols, 297
- unstructured database and, 296

Gulliver's Travels, 180

H

Hadamard, Jacques, 90

Hadamard operator, 90–91, 97.
See also operators

Hadamard qugate, 102

- basis states, 104
- initializing with, 104–105
- measurement, 164–166
- properties, 103–104
- superposition, 166–167

Hadamard transform, 259–263

Haldane, J.B.S., 346

Hamilton, Kirk, 162

hands, 21

hardware, 344–345

hedge mazes, 348

Hello, World!, 99–100

- graphical form, 100
- measuring, 162–164

Hello, XWorld!, 100–101

helpers, 231

Hilbert, David, 82

Hilbert space, 82

horizontal analysis, 133–134

horizontal systems of qugates,
 127–128

Houdini computer graphics
 system, 124

Hunt, Leigh, 157

hybrid algorithm, 126

hypothesis, 162

I

i, 32

- multiplying by, 44
- square root, 45–47

IBM Condor, 342

IBM Quantum Platform, 344

idempotent, 175

identity matrix, 88

identity operator, 88, 97.
See also operators

if-then, 126

if-then-else, 126

imaginary numbers, 32

imperfect cloning, 141

infinite set, 29

initialization

- Deutsch's algorithm, 239,
 239–240
- electronic playing cards metaphor,
 10–11
- Hadamard operator,
 104–105

inner product, 56, 66–71

- defined, 67
- projection, 69–71

integers, 31

- list of (from 0), 49
- non-negative, 31

integer-valued vectors, 50

interference, 1, 18–19, 105–107

- constructive, 18, 107
- destructive, 18, 106
- relative phase and, 199–201

interpretations, 346

inverse matrix, 93

inverse tangent, 36

invertible matrix, 93

irrational numbers, 42

J

jigsaw puzzle, 3

joint probability, 186

Jordan, Louis, 227

Jozsa, Richard, 251

Juster, Norton, 251

K

ketbra, 172

kets, 71

Kronecker, Leonard, 76

Kronecker delta, 76–77, 121

L

L1 property, 51, 85

L2 property, 52, 85

- langle, 68
- LaTeX, 345
- LEGO MINDSTORMS, 124
- Leiber, Jerry, 295
- Lennon, John, 315
- Lincoln, Abraham, 1
- linear algebra, 3, 51
- linearity, 51
 - property L1, 51
 - property L2, 51
- linearly dependent equations, 290
- linearly independent equations, 290
- linear operators, 84–87
- lists
 - ordered, 29
 - structure, 48–49
- little-endianness, 180
- local phase, 199
- loop, closed, 378

M

- magnitude, 35, 38, 95
- main diagonal, 88
- many worlds, 347
- marking stage, Grover's algorithm, 301–305
- Mastermind* game, 350
- matrix, 50
 - adjoint, 74
 - block, 144
 - column, 50, 72
 - conjugation, 92
 - degenerate, 93
 - elements, 97
 - identity, 88
 - inverse, 93
 - invertible, 93
 - linearity properties, 85
 - linear operator as, 85
 - main diagonal, 88
 - non-invertible, 93
 - projection, 174, 181
 - row, 72
 - singular, 93
 - square, 85
 - transposition, 72, 91, 91–92
- matrix elements, analysis by, 132–133
- Max system, 124

- maze solving, 348–349
- McAllister, Willy, 24
- McCartney, Paul, 315
- measurement, 1, 11, 125
 - amplitudes from projection, 171–179
- Bell states, 193–197
- Deutsch–Jozsa algorithm, 258–259, 263–264
- Deutsch's algorithm, 242–243
- electronic playing cards metaphor, 11–12
- entanglement and, 190–197
- experiments, 162–170
- Hadamard qugate, 164–166
- Hello, World!*, 162–164
- H* qugates, 164–166
- main ideas, 158–160
- meters, 161–162
- multiple qubits, 179–182
- partial collapse, 189–190
- phases, 197–201
- positive operator-valued measures, 339–340
- Postulate 4, 161
- principle of deferred measurement, 189
- principle of partial measurement, 188
- of qubits, 160–161
- tensor product, 129–130
- two *H* qugates, 166–167
- unequal superposition, 167–170
- X* qugate, 164

- Mermin, N. David, 343
- meter, 100, 158, 161–162
- mixed-product property, 138
- model, 162
- modulo, 261–262, 269, 290, 330
- multiple qubits, 179–182
- multi-qubit qugates, 131, 152–154
- multitool, 56
- multiverse, 348
- mutually exclusive, 186

N

- Napier's number, 37, 278
- natural numbers, 30

- negative numbers, 42
- Neilsen, Michael, 343
- nested sets, 30
- NISQ (noisy intermediate-scale quantum) era, 164, 341
- no-cloning theorem, 138–141
- noise, 163, 164
- nondeterministic nature of
 - measurement, 160
- non-invertible matrix, 93
- nonlinearity, 53
 - cooking example, 53
- non-negative integers, 31
- non-prime numbers, 316
- normalization, 14, 77
- NOT operator, 89, 97
- NP complexity class, 279
- numbering, 30
- number line, 42, 42
- number plane, 34
- numbers, 28–31
 - complex, 31
 - counting, 30
 - floating point, 31
 - imaginary, 32
 - natural, 30
 - real, 31
 - types of, 29–30
- number theory, 318–319

O

- objective reality, 196
- observation, 11, 158
- Occam’s razor, 346
- one-qubit qugates, 131
- one-to-one function, 282
- one-way function, 317
- operators, 84–98
 - electronic playing cards metaphor, 12–16
 - Hadamard, 90–91, 97
 - identity, 88, 97
 - linear, 84–87
 - NOT, 89, 97
 - projection, 174–175, 178
 - unitary, 94–97
- oracle complexity, 234, 313
- oracles, 229

- balanced, 242
- Bernstein–Vazirani algorithm, 270–271, 271
- classical one-bit, 230, 231
- constant, 241–242
- Deutsch–Jozsa algorithm, 252, 257–258
- Deutsch’s algorithm, 229–234, 240–242
- Grover’s algorithm, 300–310
- phase, 301
- promise, 234
- propagating input query as an output, 230
- quantum, 232–234
- querying, 240–242, 257–258
- Simon’s algorithm, 279–282
- order, 284
- ordered list, 29
- origin, 35, 57
- orthogonal, 59, 77
- orthogonal state spaces, 302
- orthonormal, 77
- outer product, 172–176

P

- Pais, Abraham, 196
- partial collapse, 189–190
- partial measurement rule, 188
- Pauli, Wolfgang, 154
- Pauli qugates, 154
- period finding, 319–322
- phase, 35, 155
 - global, 197–199
 - local, 199
 - relative, 199–201
- phase factor, 197
- phase kickback, 243, 244–245
- phase oracle, 301
- phase qugate, 155
- Plato’s allegory of the cave, 347
- Podolsky, Boris, 196
- polar coordinate system, 35
- polar form, 35
- Pong* game, 350
- positive operator-valued measures (POVMs), 339–340

- postprocessing
 - Deutsch–Jozsa algorithm, 258–259, 263–264
 - Deutsch’s algorithm, 242–243
- post-quantum cryptography, 340–341
- postulates, 2, 28
 - Postulate 1, 28
 - Postulate 2, 84, 98
 - Postulate 3, 110
 - Postulate 4, 161
- POVMs (positive operator-valued measures), 339–340
- power series, 36
- P*ugate, 155
- prediction, 162
- Preskill, John, 337
- prime factors, 316–317, 330–331
- primes, 316–318
- principle of deferred measurement, 189
- principle of partial measurement, 188
- private cards, 21
- private variable, 199
- probabilistic algorithm, 331
- probabilistic teleportation, 221–225, 223
- probabilistic values, 160
- probability, 159
 - amplitude and, 16–18
 - computing, 185–187
 - electronic playing cards metaphor, 8–9
 - joint, 186
 - negative, 16–17
- product states, 115–121
- projection, 62–65
 - amplitude from, 171–179
 - dot product, 62–65
 - inner product, 69–71
- projection matrices, 174, 181
- projection operators, 174–175, 178
- promise oracle algorithm, 238
- promise oracles, 234
- promise problem, 238
- proof by contradiction, 139
- pseudorandom, 159
- public cards, 21
- public key, 317

Q

- qcircuit, 345
- QCpy, 344
- QEC (quantum error correction), 163, 341
- QFT (quantum Fourier transform), 322, 324–326
- Qiskit, 344
- QOSF (Quantum Open Source Foundation), 344
- QPong* game, 350
- quadratic speedup, 313–314
- quantikz, 345
- quantum advantage, 342
- quantum algorithms
 - Bernstein–Vazirani, 269–276
 - Deutsch–Jozsa, 251–267
 - Deutsch’s, 227–249
 - diagrams, 124–127
 - Euclidean, 330
 - Grover’s, 295–314
 - overview, 203–205
 - Shor’s, 315–335
 - Simon’s, 277–294
 - teleportation, 207–226
- Quantum Algorithm Zoo*, 350
- Quantum Awesomeness* game, 350
- Quantum Computation and Quantum Information* (Nielsen/Chuang), 343
- Quantum Computer Science* (Mermin), 343
- quantum computing
 - applications of, 348–350
 - books, 343
 - hardware, 344–345
 - lecture notes, 343
 - online, 343–344
 - overview, 5–6
 - resources, 344–346
 - simulators, 344
 - software, 345
- quantum cryptography, 334, 340–341
- quantum encryption, 340–341
- quantum error correction (QEC), 163, 341
- quantum Fourier transform (QFT), 322, 324–326

- quantum gates. *See* qugates
- quantum identity, 274–275
- quantum mechanics
 - Copenhagen interpretation, 346
 - many-worlds interpretation, 347
 - philosophy of, 346–348
 - Postulate 1, 28
 - Postulate 2, 84
 - Postulate 3, 110
 - Postulate 4, 161
- Quantum Open Source Foundation (QOSF), 344
- quantum operations, 2
- quantum oracles, 232–234
- quantum parallelism, 204, 228, 234–248
 - circuit diagram, 235
 - conceptual illustration, 235
 - described, 236
 - oracles and, 236–237
- quantum programming, 203–205
- quantum scale, 28
- quantum states, 1, 80
 - bracket notation, 71–79
 - combining, 110–111
 - complex numbers, 32–48
 - dot product, 56–66
 - inner product, 66–71
 - numbers, 28–31
 - Postulate 2, 84
 - qubits, 79–81
 - vectors, 49–55
- quantum supremacy, 342
- qubits, 28, 79–81, 122–123
 - control, 143
 - evolution, 125
 - measurement, 160–161, 179–189, 218–219
 - multiple, 179–182
 - target, 143
- query, of function, 228
- qugates, 100–101
 - CCNOT* (*CCX*, *conditional-conditional-NOT*), 153–154
 - controlled, 151–152
 - controlled-SWAP, 153–154
 - CU*, 151–152, 152
 - CX* (*CNOT*, *conditional-NOT*, *controlled-NOT*), 141–146
 - CX'*, 153, 212, 274
 - CZ*, 154–155
 - Fredkin, 153–154
 - horizontal rules, 130–131
 - horizontal systems, 127–128
 - multi-qubit, 131, 152–154
 - one-qubit, 131
 - Pauli, 154
 - phase (*P*), 155
 - single-bit, 154–156
 - single-qubit, 131
 - SWAP, 152
 - Toffoli, 153–156
 - U*, 155
 - universal set/family, 155–156
 - vertical rules, 130–131
 - vertical systems, 128–130
 - X*, 154
 - Y*, 154
 - Z*, 154
- QuTiP, 344

R

- radius, 35–36
- random, 159
- range, 68
- Rauch, Earl Mac, 207
- ray tracing, 349
- real numbers, 31, 42
- real-valued vectors, 50
- receiver, 20. *See also* entanglement
- rectangular coordinate system, 35
- rectangular diagram, 35
- reflections, 298–300
- register, 254
- relatively prime numbers, 330
- relative phase, 199–201
- repeat length, 319
- retrocausal, 347
- reversible matrices, 97
- right-hand rule, 348
- Rivest, Ron, 317
- Robbins, Tom, 109
- Rosen, Nathan, 196
- row matrix, 72
- RSA cryptography, 317

S

scalar multiplication, 50
Scratch, 124
self-inverse, 231
sender, 20. *See also* entanglement
Seneca the Younger, 203
sequences, 29
serial action, 127
sets, 29–30
 basis, 54
 finite, 29
 infinite, 29
 nested, 30
Shamir, Adi, 317
Shor's algorithm, 315–335. *See also*
 quantum algorithms
 circuit analysis, 322–331
 example of, 331–333
 overview, 297–298, 318–319
 period finding, 319–322
 prime factoring, 330–331
 primes and encryption, 316–318
 probability of k , 324–326
 quantum Fourier transform,
 324–326
 step by step, 330
shot, 163
sift, 77
Simon, Daniel, 279
Simon's algorithm, 277–294. *See also*
 quantum algorithms
 vs. classical algorithm, 293–294, 294
 classical solution, 283–284
 combining quantum outputs,
 289–291
 example, 291–292
 exponential speedup, 278, 294
 oracle, 279–282
 quantum part of, 285, 285–287
 rewriting x and measuring,
 287–289
 secret string, 291–292, 292
 without measurement of y , 291
simulators, 344
single-bit qugates, 154–156
single-qubit qugates, 131
singular matrix, 93
software, 344

Spivery, William, 83
squared amplitude, 7
square matrix, 85
Stack Exchange, 346
states, 80. *See also* quantum algorithms
 Bell. *See* Bell states
 computational basis, 118
 electronic playing cards metaphor,
 7–9
 entangled, 148
 measurement of, 161
 product, 115–121
 teleportation, 210–211
 tensor product, 115–121
state vector, 80
Stoller, Mike, 295
superdense coding, 338, 338–339
superluminal transport, 192.
 See also entanglement
superposition, 1, 101–105.
 See also entanglement
 Bell states, 191–192
 collapse, 11, 12
 electronic playing cards metaphor,
 9–10
 equal, 10
 partially collapsed, 191
 shared, 21
 unequal, 167–170
 uniform, 10, 255
Susskind, Leonard, 49
SWAP qugates, 152
switch, 142–146
systems
 circuit analysis, 131–138
 CX qugate, 141–146
 defined, 109
 no-cloning theorem, 138–141
 Postulate 3, 110
 quantum algorithm diagrams,
 124–127
 qugates, 127–131
 tensor product, 111–115

T

tableau form, 52
Tagore, Rabindranath, 157
target, 142

- target qubit, 143
- teleportation, 207–226. *See also* quantum algorithms
 - algorithm, 210
 - probabilistic, 221–225, 223
 - process, 212–220
 - properties of, 208
 - protocol, 221, 221
 - quantum versus fictional, 208
 - qubit measurement, 218–219
 - signal, 209
 - state, 210–211
 - thought experiment, 209–210
- tensor product, 110, 111–115, 129–130
- tensor product states, 115–121
- testing variables, 126
- theoretical mathematics, 316
- theory, 162
- thought experiment, 209–210
- Toffoli qugate, 153–156
- transposition, 91, 91–92
- traversal, 292
- triple modular redundancy, 163
- two-to-one function, 281

U

- U qugate, 155
- umwelt, 267
- uncorrelated noise, 164, 186
- unequal superposition, 167–170
- uniform superposition, 10, 255
- unique product, 317
- unit arrow, 63
- unitary operators, 94–97
- unit magnitude, 63, 65, 77, 80, 95
- unit norm, 77
- unit vectors, 63
- universal qugates, 155
- universal set/family of qugates, 155–156
- unstructured database, 295–296

V

- V1 property, 50
- V2 property, 51
- van Gogh, Vincent, 27
- variables
 - private, 199
 - testing, 126

- Vazirani, Umesh, 115, 270
- vector addition, 50
- vectors, 49–55. *See also* quantum states
 - adjoint, 74
 - basis, 48, 54
 - coefficient, 55
 - complex-valued, 50, 66
 - conjugated transpose, 74
 - coordinate, 55
 - dimensionality of, 50
 - integer-valued, 50
 - linearity, 51–54
 - real-valued, 50
 - unit, 63
- vector space, 82
- vertical analysis, 134–137
- vertical systems of qugates, 128–130
- von Neumann, John, 5
- von Neumann machines, 5–6

W

- Waits, Tom, 41
- Weinersmith, Zach, 5
- write-once, 7

X

- X1 property, 232
- X2 property, 232
- X3 property, 232
- XOR, 231, 262, 280
 - bitstring, 280
 - properties, 232
- Xqugate, 154, 164

Y

- yquant, 345
- Yqugate, 154

Z

- zero, 42
- Zqugate, 154
- ZX-calculus, 342
- ZX-diagrams, 342